

**FACULDADE DE TECNOLOGIA “PADRE DANILO JOSÉ DE OLIVEIRA OHL” –
FATEC BARUERI**

CURSO SUPERIOR EM GESTÃO DA TECNOLOGIA DA INFORMAÇÃO

**EDUARDO GARCIA DANTAS
GEOVANA LOPES DE SOUSA
JOÃO VICTOR CEZARINO LOPES
LEANDRO DE OLIVEIRA PEREIRA
VINICIUS MONTEIRO DOS SANTOS**

**PROJETO INTERDISCIPLINAR IV – PROGRAMAÇÃO PARA INTERNET –
CRIAÇÃO DE SITE PARA A OGGI BARUERI (CENTRO)**

BARUERI

2026-1

**FACULDADE DE TECNOLOGIA “PADRE DANILO JOSÉ DE OLIVEIRA OHL” –
FATEC BARUERI**

CURSO SUPERIOR EM GESTÃO DA TECNOLOGIA DA INFORMAÇÃO

**EDUARDO GARCIA DANTAS
GEOVANA LOPES DE SOUSA
JOÃO VICTOR CEZARINO LOPES
LEANDRO DE OLIVEIRA PEREIRA
VINICIUS MONTEIRO DOS SANTOS**

**PROJETO INTERDISCIPLINAR IV – PROGRAMAÇÃO PARA INTERNET –
CRIAÇÃO DE SITE PARA A OGGI BARUERI (CENTRO)**

Projeto da disciplina Projeto Interdisciplinar IV em Gestão da tecnologia da informação, apresentado à Faculdade de Tecnologia Padre Danilo José de Oliveira Ohl – Fatec Barueri.

Orientador: Prof. Vander Ribeiro Elme

BARUERI

2026-1

FICHA CATALOGRÁFICA

Ficha catalográfica elaborada pelos autores:

Eduardo Garcia Dantas / Geovana Lopes de Sousa / João Victor Cezarino Lopes /
Leandro de Oliveira Pereira / Vinicius Monteiro dos Santos

Sistema de gerenciamento de reservas de carrinhos de sorvete para a filial da empresa
Oggi (Centro de Barueri) – 2026.

Trabalho de Conclusão de Curso (Graduação em Gestão da Tecnologia da Informação)
– Faculdade de Tecnologia de Barueri, Barueri, 2026.

Orientador: Vander Ribeiro Elme

1. Sistemas web. 2. Reservas. 3. Django. 4. Banco de dados. 5. Desenvolvimento de
software. I. Título.

TERMO DE APROVAÇÃO

Trabalho de Graduação apresentado à Faculdade de Tecnologia de Barueri – FATEC Barueri, como requisito parcial para obtenção do título de Tecnólogo em Gestão da Tecnologia da Informação.

Título do Trabalho:

PROJETO INTERDISCIPLINAR IV – PROGRAMAÇÃO PARA INTERNET – CRIAÇÃO DE SITE PARA A OGGI BARUERI (CENTRO)

Autores:

- EDUARDO GARCIA DANTAS
- GEOVANA LOPES DE SOUSA
- JOÃO VICTOR CEZARINO LOPES
- LEANDRO DE OLIVEIRA PEREIRA
- VINICIUS MONTEIRO DOS SANTOS

Trabalho aprovado em: __ / __ / __

Banca Examinadora

Prof. Me. Vander Ribeiro Elme

Faculdade de Tecnologia de Barueri – FATEC Barueri

Professor(a) Examinador(a)

Professor(a) Examinador(a)

Barueri – SP

2026

Agradecimentos

Agradecemos especialmente ao Scrum Master Leandro de Oliveira Pereira, pela liderança, organização e apoio durante todas as etapas do desenvolvimento do projeto, contribuindo significativamente para o alinhamento da equipe e para o bom andamento das atividades realizadas.

Agradecemos também ao Lucas e aos demais colaboradores da empresa Oggi Sorvetes, pela disponibilidade, atenção e colaboração fornecidas ao longo do desenvolvimento deste trabalho, contribuindo com informações importantes para a compreensão das necessidades reais da empresa.

Ao Prof. Vander Ribeiro Elme, pela orientação, suporte e compartilhamento de conhecimento durante o desenvolvimento do projeto, auxiliando na construção acadêmica e técnica da equipe.

À equipe da Faculdade de Tecnologia de Barueri – FATEC Barueri, pela oportunidade de aprendizado, estrutura oferecida e apoio ao longo da formação acadêmica.

RESUMO

Este trabalho apresenta o desenvolvimento de um sistema web para gerenciamento de reservas de carrinhos de sorvete, com foco na organização de pedidos, controle de disponibilidade e melhoria da experiência do usuário. O sistema permite que clientes realizem solicitações de reserva, escolham sabores e acompanhem o status do pedido, enquanto o administrador gerencia carrinhos, produtos e confirmações.

A aplicação foi desenvolvida utilizando conceitos de arquitetura MVT (Model-View-Template) do framework Django, além do uso de ORM (Object-Relational Mapping) para manipulação do banco de dados. Também foram implementadas regras de negócio para controle de disponibilidade, limite de reservas por data e validação de pedidos.

Como resultado, o sistema proporciona maior eficiência na organização das reservas, reduzindo erros manuais e otimizando o processo de atendimento. Além disso, contribui para a digitalização do serviço, tornando-o mais acessível e estruturado.

ABSTRACT

This project presents the development of a web system for managing ice cream cart reservations, focusing on order organization, availability control, and user experience improvement. The system allows customers to request reservations, select flavors, and track the order status, while the administrator manages carts, products, and confirmations.

The application was developed using the MVT (Model-View-Template) architecture of the Django framework, along with the use of ORM (Object-Relational Mapping) for database manipulation. Business rules were also implemented to control availability, reservation limits per date, and order validation.

As a result, the system provides greater efficiency in managing reservations, reducing manual errors and optimizing the service process. In addition, it contributes to service digitalization, making it more accessible and structured.

RESUMEN

Este trabajo presenta el desarrollo de un sistema web para la gestión de reservas de carritos de helado, con enfoque en la organización de pedidos, control de disponibilidad y mejora de la experiencia del usuario. El sistema permite a los clientes realizar solicitudes de reserva, elegir sabores y seguir el estado del pedido, mientras que el administrador gestiona carritos, productos y confirmaciones.

La aplicación fue desarrollada utilizando la arquitectura MVT (Model-View-Template) del framework Django, además del uso de ORM (Object-Relational Mapping) para la manipulación de la base de datos. También se implementaron reglas de negocio para el control de disponibilidad, límite de reservas por fecha y validación de pedidos.

Como resultado, el sistema proporciona mayor eficiencia en la organización de las reservas, reduciendo errores manuales y optimizando el proceso de atención. Además, contribuye a la digitalización del servicio, haciéndolo más accesible y estructurado.

LISTA DE ABREVIATURAS E SIGLAS

ABNT – Associação Brasileira de Normas Técnicas

API – Application Programming Interface (Interface de Programação de Aplicações)

CRUD – Create, Read, Update and Delete (Criar, Ler, Atualizar e Excluir)

CSS – Cascading Style Sheets (Folhas de Estilo em Cascata)

DER – Diagrama Entidade Relacionamento

ERP – Enterprise Resource Planning (Planejamento dos Recursos Empresariais)

FK – Foreign Key (Chave Estrangeira)

HTML – HyperText Markup Language (Linguagem de Marcação de Hipertexto)

HTTPS – HyperText Transfer Protocol Secure (Protocolo Seguro de Transferência de Hipertexto)

LGPD – Lei Geral de Proteção de Dados

MER – Modelo Entidade Relacionamento

MVC – Model-View-Controller (Modelo-Visão-Controlador)

MVT – Model-View-Template (Modelo-Visão-Template)

ORM – Object-Relational Mapping (Mapeamento Objeto-Relacional)

PK – Primary Key (Chave Primária)

RF – Requisito Funcional

RN – Regra de Negócio

RNF – Requisito Não Funcional

SGBD – Sistema Gerenciador de Banco de Dados

SQL – Structured Query Language (Linguagem de Consulta Estruturada)

TI – Tecnologia da Informação

UML – Unified Modeling Language (Linguagem de Modelagem Unificada)

URL – Uniform Resource Locator (Localizador Uniforme de Recursos)

WhatsApp – Aplicativo de mensagens instantâneas para comunicação

SUMÁRIO

1. INTRODUÇÃO

A tecnologia da informação tem se tornado cada vez mais presente no cotidiano das pessoas e das organizações, influenciando diretamente a forma como serviços são prestados e gerenciados. Sistemas digitais vêm sendo amplamente utilizados para facilitar processos que antes eram realizados manualmente, proporcionando maior agilidade, organização e eficiência. No contexto comercial, especialmente no setor de vendas e locação de produtos, a utilização de sistemas informatizados contribui significativamente para o controle de estoque, gestão de pedidos e atendimento ao cliente.

No caso da empresa Oggi, que atua na comercialização de sorvetes e disponibilização de carrinhos para eventos e vendas, a adoção de um sistema digital pode trazer melhorias significativas na gestão de reservas e no controle dos produtos disponíveis. Atualmente, a ausência de um sistema estruturado pode ocasionar dificuldades no gerenciamento das informações, como falhas na verificação de disponibilidade de carrinhos, inconsistências no controle de produtos e possíveis erros no atendimento ao cliente.

Diante desse cenário, torna-se necessário o desenvolvimento de um sistema voltado para o gerenciamento de reservas de carrinhos de sorvete da empresa Oggi, permitindo que clientes consultem a disponibilidade, selecionem produtos e realizem pedidos de forma prática e organizada. Além disso, o sistema possibilitará um melhor controle interno, contribuindo para a eficiência operacional e a confiabilidade das informações. Dessa forma, o presente projeto propõe a criação de uma solução tecnológica que auxilie na modernização dos processos da empresa, promovendo melhorias tanto na gestão quanto na experiência do cliente.

1.1. OBJETIVOS

Objetivo geral: Desenvolver um **protótipo** de sistema web para a locação de carrinhos de sorvete para festas e eventos, em parceria com a empresa Oggi Sorvetes do centro de Barueri. A proposta é criar uma plataforma simples e intuitiva, onde o cliente possa visualizar os carrinhos disponíveis, consultar sabores, verificar datas livres e realizar reservas de forma prática.

Objetivos específicos:

- Desenvolver uma interface que permita ao usuário visualizar carrinhos e produtos disponíveis para reserva;
- Implementar um sistema de verificação de disponibilidade de carrinhos e produtos no banco de dados;

- Permitir que o usuário selecione produtos e realize a reserva pelo sistema;
- Criar um banco de dados para armazenar informações de clientes, reservas, carrinhos e produtos;
- Desenvolver funcionalidades para confirmação e gerenciamento das reservas realizadas;
- Garantir a organização e integridade dos dados armazenados no sistema;
- Documentar a arquitetura do sistema utilizando diagramas UML, como diagrama de classes, sequência, colaboração e estados;
- Aplicar boas práticas de desenvolvimento e organização do código no frontend e backend;
- Implementar medidas básicas de proteção de dados dos usuários conforme princípios da LGPD.

1.2. JUSTIFICATIVA

Justifica-se o desenvolvimento deste projeto como uma forma de estudo e compreensão sobre a aplicação de sistemas informatizados no gerenciamento de reservas e controle de produtos em ambientes comerciais. Acredita-se que a utilização de soluções tecnológicas pode contribuir significativamente para a organização de processos, redução de erros operacionais e melhoria na experiência do cliente.

Além disso, torna-se relevante analisar como a implementação de um sistema digital pode impactar positivamente a gestão de serviços, especialmente no contexto da empresa Oggi, que realiza a locação de carrinhos de sorvete e comercialização de produtos. A ausência de ferramentas adequadas pode gerar dificuldades no controle de disponibilidade, inconsistências nas informações e limitações no atendimento ao cliente, o que reforça a necessidade de uma solução estruturada.

Dessa forma, este projeto busca desenvolver uma solução que não apenas atenda às necessidades operacionais da empresa, mas também contribua para a modernização de seus processos, promovendo maior eficiência, organização e confiabilidade. Ao mesmo tempo, possibilita a aplicação prática dos conhecimentos adquiridos na área de tecnologia da informação, fortalecendo a formação acadêmica e profissional dos envolvidos.

1.3. METODOLOGIA

A metodologia adotada neste projeto baseia-se em uma abordagem incremental,

utilizando conceitos de desenvolvimento ágil para a construção do sistema de reservas de carrinhos de sorvete da empresa Oggi. Inicialmente, realiza-se o levantamento de requisitos, identificando as principais funcionalidades do sistema, como consulta de disponibilidade, seleção de produtos e gerenciamento de reservas.

Em seguida, são elaborados diagramas e modelos, incluindo diagramas UML e modelagem de banco de dados (conceitual, lógico e físico), com o objetivo de estruturar o sistema de forma organizada. Também são aplicados padrões de desenvolvimento, como MVC, para organização da aplicação, e ORM, para integração com o banco de dados.

O desenvolvimento ocorre por meio de sprints, permitindo a implementação gradual das funcionalidades do frontend e backend, com realização de testes e ajustes contínuos. Por fim, são elaboradas as documentações do sistema e considerados aspectos de segurança e proteção de dados, conforme os princípios da LGPD.

2. HISTÓRICO

2.1. CONTEXTUALIZAÇÃO

O projeto foi idealizado a partir da necessidade de organizar e automatizar o processo de reservas de carrinhos de sorvete, que atualmente ocorre de forma manual, via mensagens no WhatsApp, dificultando o controle de disponibilidade, organização de pedidos e gestão de confirmações.

- A ausência de um sistema estruturado pode ocasionar:
- Conflito de datas
- Excesso de reservas acima da capacidade disponível
- Falta de padronização das informações do cliente
- Dificuldade no controle de status das reservas
- Diante desse cenário, identificou-se a oportunidade de desenvolver uma solução digital que centralize e organize o processo de solicitação e gerenciamento de reservas.

2.2. EVOLUÇÃO DAS IDEIAS

O desenvolvimento do projeto seguiu as seguintes etapas:

- Levantamento de requisitos junto à necessidade operacional do negócio;
- Definição das regras de negócio relacionadas à capacidade diária e confirmação de reservas;

- Modelagem conceitual do banco de dados (DER);
- Definição dos requisitos funcionais, não funcionais e de negócio;
- Estruturação das regras de controle de disponibilidade e concorrência.

A proposta evoluiu de um simples sistema de solicitação para uma solução com controle de status, regras de confirmação

3. REQUISITOS

3.1. REQUISITOS FUNCIONAIS

RF01 – Consulta de disponibilidade

O sistema deve exibir um calendário com a disponibilidade de carrinhos por data para solicitação de reserva;

RF02 – Seleção de data para reserva

O sistema deve permitir que o cliente selecione apenas datas disponíveis para realizar a solicitação de reserva;

RF03 – Cardápio de produtos

O sistema deve exibir o cardápio de sabores de sorvete disponíveis para seleção pelo cliente;

RF04 – Montagem da solicitação

O sistema deve permitir que o cliente selecione sabores, quantidades e visualize o valor total da solicitação;

RF05 – Dados do cliente

O sistema deve permitir o preenchimento dos dados do cliente, incluindo nome, telefone (WhatsApp obrigatório), endereço e observações;

RF06 – Envio da solicitação

O sistema deve permitir o envio da solicitação de reserva, exigindo aceite dos termos, registrando data/hora e criando a reserva com status PENDENTE;

RF07 – Contato com a empresa

O sistema deve gerar automaticamente um link de contato via WhatsApp após o envio da solicitação;

RF08 – Restrição de edição

O sistema deve impedir que o cliente edite a solicitação após o envio;

RF09 – Acesso administrativo

O sistema deve permitir a autenticação do administrador para acesso às funcionalidades administrativas;

RF10 – Gerenciamento de reservas

O sistema deve permitir ao administrador visualizar, confirmar, ajustar, recusar ou cancelar reservas, respeitando a disponibilidade de carrinhos;

RF11 – Controle de carrinhos

O sistema deve permitir a associação de reservas confirmadas a carrinhos, garantindo o limite diário disponível;

RF12 – Gerenciamento de produtos

O sistema deve permitir o gerenciamento do cardápio de sabores pelo administrador;

RF13 – Consulta de detalhes

O sistema deve permitir a visualização dos dados completos da reserva, incluindo cliente, itens e status;

3.2. REQUISITOS NÃO FUNCIONAIS

RNF01 – Usabilidade:

O sistema deve ser responsivo (mobile-first) [Objeto], adaptando-se a diferentes tamanhos de tela [Condição];

RNF02 – Desempenho:

O sistema deve garantir tempo de resposta inferior a 2 segundos [Objeto] para operações comuns [Critério];

RNF03 – Segurança de senhas:

O sistema deve armazenar senhas de forma segura [Objeto] para proteção dos dados do administrador [Finalidade];

RNF04 – Segurança contra ataques:

O sistema deve possuir proteção contra SQL Injection [Objeto] por meio de validação e prepared statements [Critério];

RNF05 – Confiabilidade:

O sistema deve garantir disponibilidade mínima de 99% [Objeto] durante o período de funcionamento [Critério];

RNF06 – Validação de dados:

O backend deve validar todos os dados recebidos do cliente [Objeto] antes do processamento [Critério];

3.3. REQUISITOS DE NEGÓCIO

RN01 – Capacidade diária:

O sistema deve considerar que existem 3 carrinhos disponíveis [Regra geral];

RN02 – Limite de confirmações:

O sistema deve permitir o máximo da quantidade cadastrada de reservas com status CONFIRMADO por data [Critério];

RN03 – Reserva pendente:

Reservas com status PENDENTE não ocupam carrinho [Regra];

RN04 – Ocupação de carrinho:

Apenas reservas com status CONFIRMADO ocupam carrinho [Condição];

RN05 – Associação:

Um carrinho pode estar associado a apenas uma reserva por data, porém uma reserva pode conter múltiplos carrinhos [Regra];

RN06 – Indisponibilidade de data:

Uma data deve ser considerada indisponível quando atingir 3 confirmações [Critério];

RN07 – Confirmação oficial:

A confirmação oficial da reserva ocorre exclusivamente via WhatsApp [Canal];

RN08 – Aceite obrigatório:

O cliente deve aceitar os termos e condições antes da solicitação [Regra];

RN09 – Status válidos:

A reserva deve possuir apenas os seguintes status válidos: confirmado, pendente e cancelado sendo permitido o controle individual por reserva contendo múltiplos carrinhos [Restrição];

RN10 – Alteração administrativa:

O administrador pode alterar o pedido antes da confirmação [Permissão];

RN11 – Controle de concorrência:

O sistema deve impedir confirmações simultâneas acima da capacidade diária [Regra];

RN12 – Reserva duplicada:

Um cliente pode possuir mais de uma reserva na mesma data, desde que a capacidade máxima de carrinhos disponíveis não seja excedida [Restrição];

RN13 – Valor da diária

O sistema deve informar que o valor da locação do carrinho será de R\$ 50,00 por dia [Regra];

RN14 - Isenção da taxa do carrinho por volume de compra

O valor da diária do carrinho não será cobrado caso o cliente adquira R\$ 300,00 ou mais em sorvetes na mesma reserva [Critério];

3.4. NÃO ESCOPO DO SISTEMA

NE01 – Sistema de pagamento online:

O sistema não realizará processamento de pagamentos via cartão de crédito;

NE02 – Controle logístico de entrega:

O sistema não será responsável pelo transporte, entrega ou retirada física do carrinho;

NE03 – Gestão financeira completa:

O sistema não realizará controle de fluxo de caixa, emissão de notas fiscais ou relatórios contábeis;

NE04 – Aplicativo mobile nativo:

O sistema não será desenvolvido como aplicativo Android ou iOS, sendo disponibilizado apenas como aplicação web responsiva;

NE05 – Integração com sistemas externos:

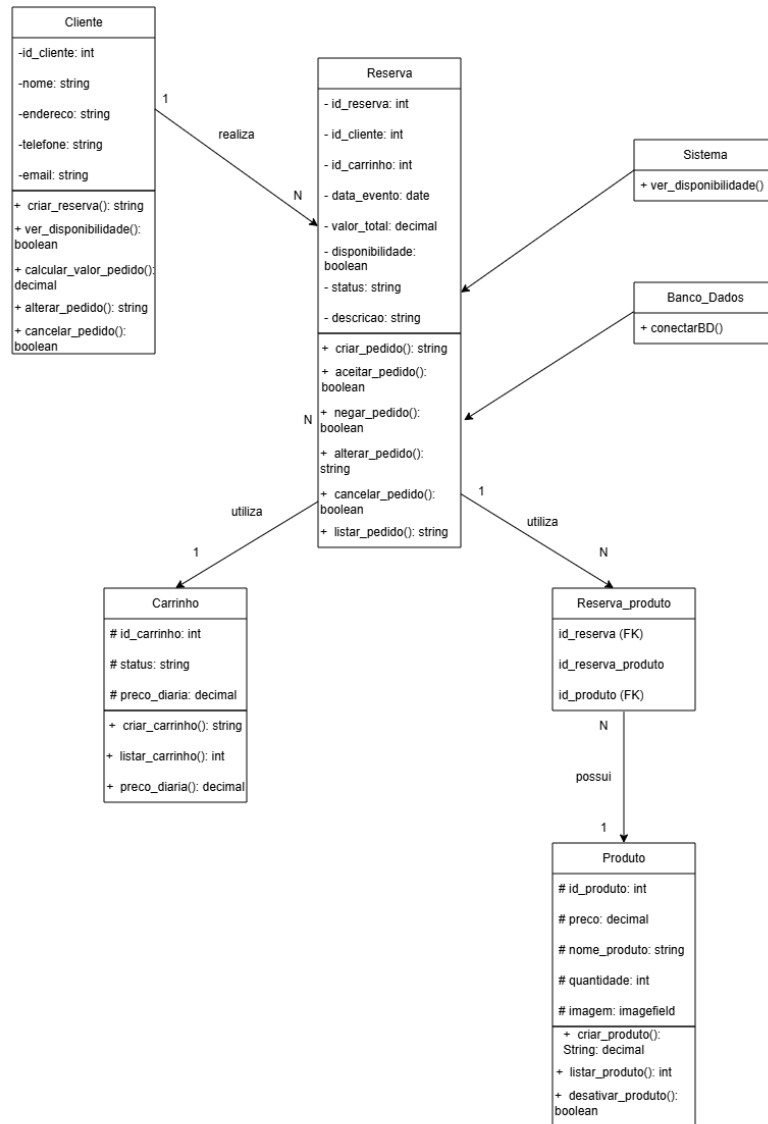
O sistema não possuirá integração automática com sistemas de ERP, contabilidade ou plataformas externas;

NE06 – Chat automatizado:

O sistema não realizará atendimento automático ao cliente, limitando-se à geração de link para contato via WhatsApp.

4. DIAGRAMAS

4.1. DIAGRAMA DE CLASSES



4.1.1. Descrição Geral

Guedes (p. 31) explica que o diagrama de classes define a estrutura das classes presentes no sistema, especificando seus atributos e métodos, além de demonstrar como essas classes se relacionam e trocam informações entre si.

O diagrama de classes apresentado descreve a estrutura do sistema de reserva de carrinhos de sorvete, evidenciando as principais classes, seus atributos, métodos e os relacionamentos entre elas. Ele representa de forma estática como os dados e funcionalidades estão organizados, servindo como base para a implementação do sistema.

O modelo contempla entidades como cliente, reserva, carrinho e produto, além de

componentes auxiliares como sistema e banco de dados. Também são apresentadas as relações entre essas classes, incluindo associações e cardinalidades.

4.1.2. Classes Participantes

Cliente: Representa o usuário do sistema. Possui informações como id, nome, endereço, telefone e e-mail. Também contém métodos para criar, visualizar, alterar e cancelar pedidos, além de verificar disponibilidade.

Reserva: Classe central do sistema, responsável por representar o pedido realizado pelo cliente. Contém dados como id da reserva, id do cliente, id do carrinho, data do evento, valor total, status e descrição. Também possui métodos para criar, aceitar, negar, alterar, cancelar e listar pedidos.

Carrinho: Representa o carrinho de sorvete disponível para reserva. Possui atributos como id, status e preço diário. Inclui operações para criação, listagem e consulta de preço.

Produto: Representa os itens disponíveis para o carrinho. Possui atributos como id, nome, preço, imagem e quantidade. Também inclui métodos para criação, listagem e desativação de produtos.

Reserva_Produto: Classe associativa que representa o relacionamento entre reserva e produto, permitindo que uma reserva tenha vários produtos. Contém chaves estrangeiras para reserva e produto.

Sistema: Responsável por fornecer funcionalidades gerais, como a verificação de disponibilidade.

Banco_Dados: Responsável pela persistência dos dados, contendo a operação de conexão com o banco.

4.1.3. Relacionamentos entre as Classes

Cliente → Reserva (1:N):

Um cliente pode realizar várias reservas, mas cada reserva pertence a apenas um cliente.

Reserva → Carrinho (N:1):

Uma reserva utiliza um carrinho, enquanto um carrinho pode estar associado a várias reservas ao longo do tempo.

Reserva → Reserva_Produto (1:N):

Uma reserva pode conter vários itens (produtos), representados pela classe intermediária.

Reserva_Produto → Produto (N:1):

Cada item da reserva está associado a um único produto, mas um produto pode aparecer

em várias reservas.

Reserva → Sistema:

A reserva interage com o sistema para verificar disponibilidade.

Reserva → Banco_Dados:

A reserva utiliza o banco de dados para armazenar e recuperar informações.

4.1.4. Observações sobre a Estrutura

A classe Reserva é o núcleo do sistema, centralizando as principais operações.

A classe Reserva_Produto resolve o relacionamento muitos-para-muitos entre reserva e produto.

Há separação clara entre lógica de negócio (classes principais) e infraestrutura (Sistema e Banco de Dados).

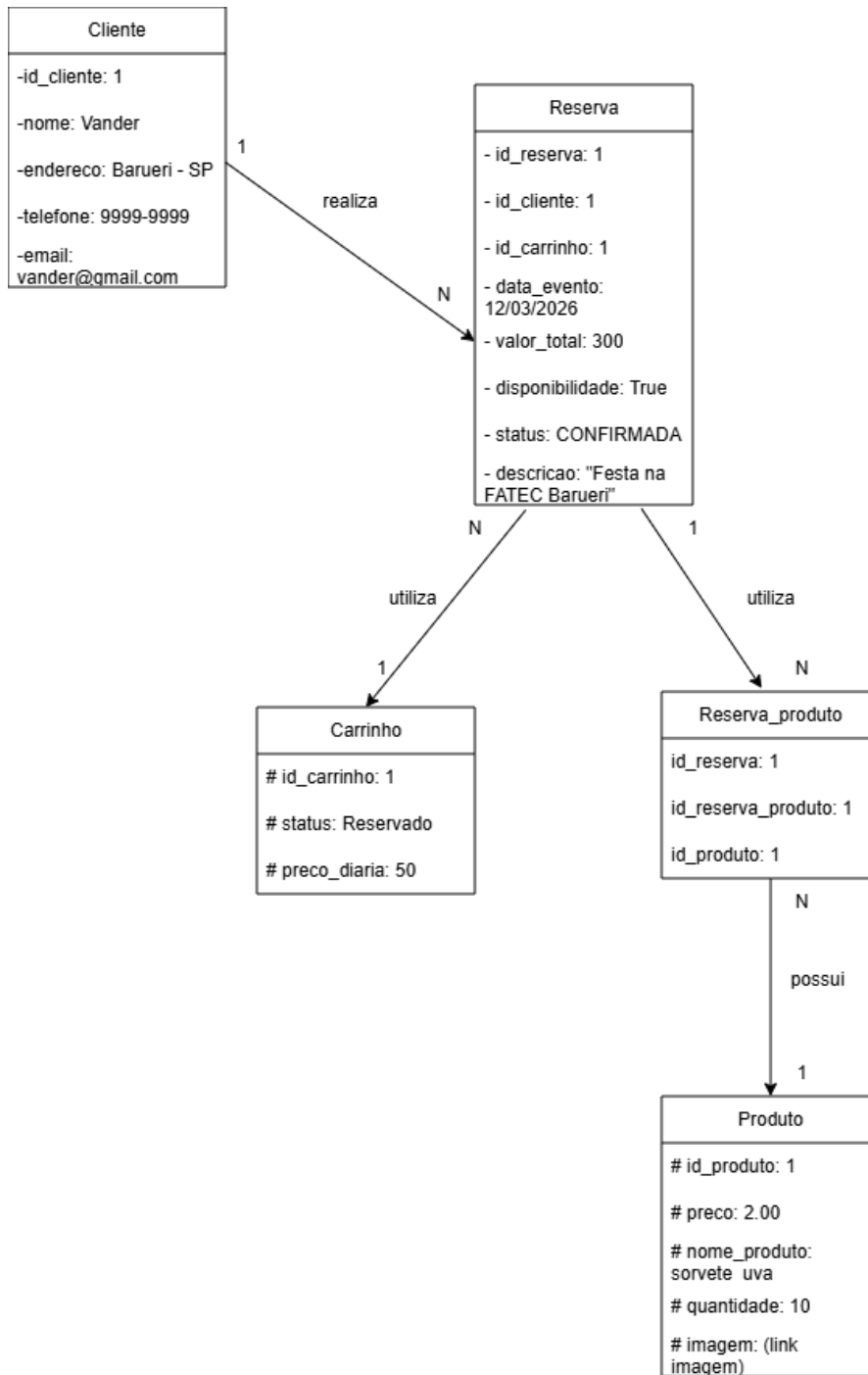
Os métodos definidos nas classes indicam as principais funcionalidades disponíveis no sistema.

4.1.5. Considerações Finais

O diagrama de classes permite compreender de forma clara a organização estrutural do sistema, mostrando como as entidades se relacionam e quais responsabilidades cada uma possui. Essa representação é essencial para o desenvolvimento, pois facilita a implementação orientada a objetos e garante maior organização e reutilização de código.

Além disso, o modelo evidencia boas práticas, como a separação de responsabilidades e o uso de classes intermediárias para relacionamentos complexos, contribuindo para a escalabilidade e manutenção do sistema.

4.2. DIAGRAMA DE OBJETOS



4.2.1. Descrição Geral

Segundo Guedes (p.32), o diagrama de objetos está diretamente relacionado ao diagrama de classes, sendo considerado um complemento deste. Ele fornece uma visão instantânea dos valores atribuídos aos objetos em um determinado momento da execução do sistema, permitindo compreender como as instâncias das classes se comportam e se organizam em uma situação específica.

O diagrama de objetos representa a estrutura de dados do sistema de reserva de carrinhos de sorvete, descrevendo as entidades principais, seus atributos e os relacionamentos existentes entre elas.

Esse modelo define como as informações são organizadas e armazenadas, servindo como base para a implementação do banco de dados e para o funcionamento das regras de negócio do sistema.

4.2.2. Descrição das Entidades

O sistema é composto por cinco entidades principais: Cliente, Reserva, Carrinho, Produto e Reserva_Produto.

A entidade Cliente é responsável por armazenar os dados dos usuários que utilizam o sistema. Cada cliente possui um identificador único, além de informações como nome, endereço, telefone e e-mail. Esses dados são essenciais para identificar o responsável por cada reserva realizada.

A entidade Reserva representa o elemento central do sistema, pois registra todas as reservas feitas pelos clientes. Cada reserva contém informações como a data do evento, o valor total, a disponibilidade, o status e uma descrição. Além disso, a reserva possui referências ao cliente que a realizou e ao carrinho utilizado, garantindo a ligação entre as entidades.

A entidade Carrinho armazena os dados dos carrinhos de sorvete disponíveis para locação. Cada carrinho possui um identificador, um status (como disponível ou reservado) e um valor de diária. Essa entidade é fundamental para controlar a disponibilidade dos recursos oferecidos pelo sistema.

A entidade Produto representa os itens que podem ser associados a uma reserva, como os tipos de sorvete disponíveis. Cada produto possui um identificador, nome, preço e quantidade em estoque, permitindo o controle dos itens oferecidos aos clientes.

Por fim, a entidade Reserva_Produto atua como uma entidade intermediária, responsável por relacionar as reservas com os produtos. Ela permite que uma única reserva esteja associada a vários produtos e que um produto possa fazer parte de várias reservas, caracterizando um relacionamento do tipo muitos-para-muitos.

4.2.3. Relacionamentos entre as Entidades

Os relacionamentos entre as entidades definem como os dados se conectam dentro do sistema.

Um cliente pode realizar várias reservas, mas cada reserva está associada a apenas um cliente. Isso caracteriza um relacionamento do tipo um-para-muitos entre Cliente e Reserva.

Cada reserva utiliza um único carrinho, enquanto um carrinho pode ser utilizado em

várias reservas ao longo do tempo. Esse relacionamento também é do tipo um-para-muitos.

Além disso, uma reserva pode conter vários produtos, e um produto pode estar presente em diversas reservas. Para viabilizar esse relacionamento muitos-para-muitos, é utilizada a entidade intermediária Reserva_Produto.

4.2.4. Regras de Negócio Representadas

O modelo também evidencia algumas regras importantes do sistema.

Uma reserva só pode existir se estiver associada a um cliente válido. Da mesma forma, é necessário que exista um carrinho disponível para que a reserva seja realizada.

Os produtos vinculados a uma reserva são controlados por meio da entidade intermediária, garantindo flexibilidade e organização no armazenamento dos dados.

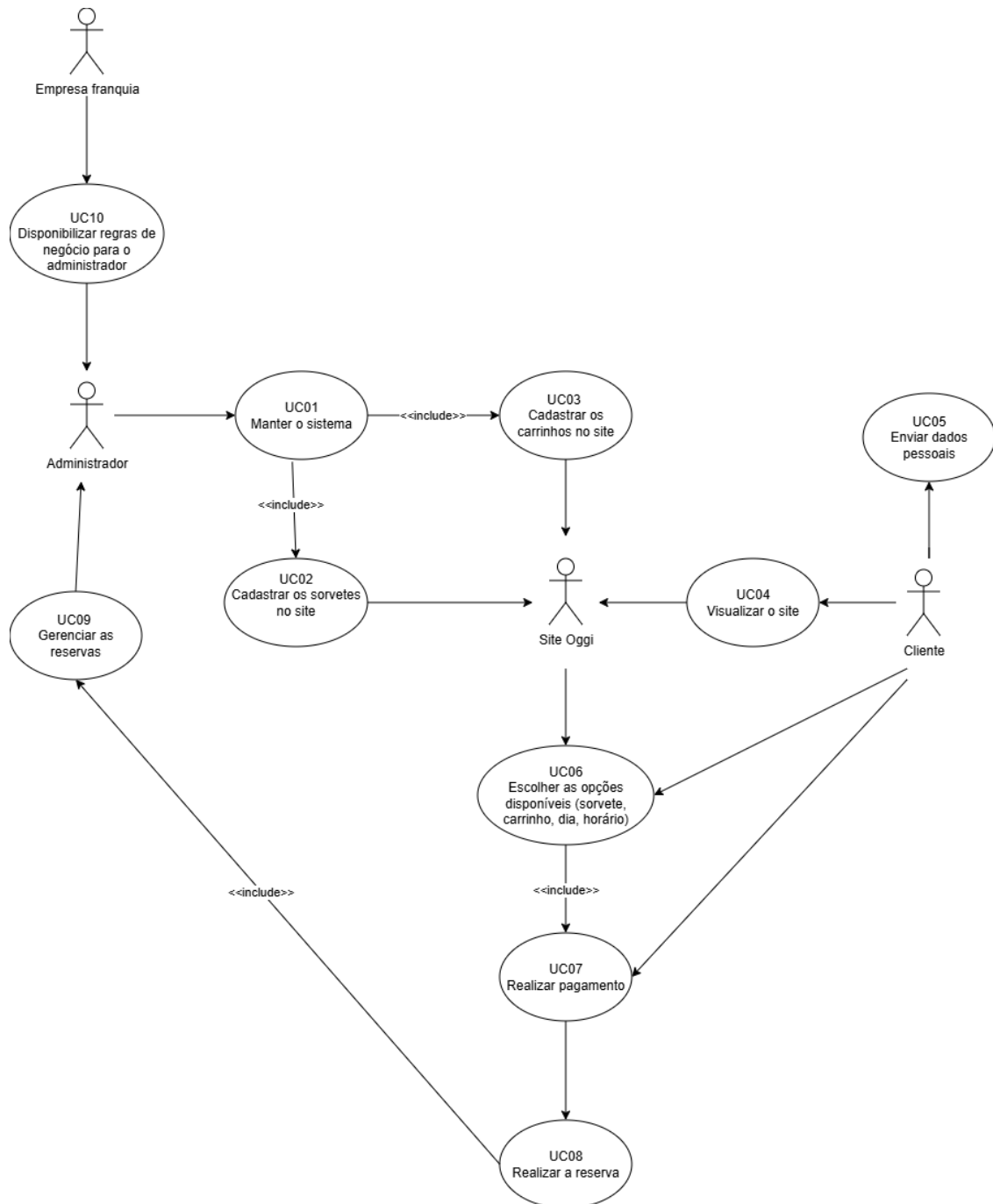
Além disso, o status da reserva permite identificar sua situação atual, como confirmada, pendente ou cancelada, o que é essencial para o gerenciamento do sistema.

4.2.5. Considerações Finais

O diagrama objeto fornece uma visão clara e estruturada da organização dos dados no sistema, facilitando tanto o desenvolvimento quanto a manutenção da aplicação.

Sua utilização permite garantir consistência nas informações, além de apoiar a implementação correta das funcionalidades relacionadas às reservas, produtos e clientes.

4.3. DIAGRAMA DE CASO DE USO



4.3.1. Descrição Geral

Segundo Guedes (p. 30), o diagrama de caso de uso é considerado o mais geral e informal da UML, por utilizar uma linguagem simples e acessível, permitindo que usuários compreendam de forma clara o comportamento geral do sistema.

O diagrama de casos de uso apresenta as funcionalidades principais do sistema de reserva de carrinhos de sorvete, destacando as interações entre os atores e o sistema. Ele evidencia como administradores, clientes e a empresa franqueadora utilizam o sistema para gerenciar produtos, reservas e operações do site.

4.3.2. Atores

Empresa Franquia: responsável por definir regras de negócio que serão utilizadas no sistema.

Administrador: gerencia o sistema, incluindo cadastro de produtos e carrinhos, além do controle de reservas.

Cliente: usuário final que acessa o site para visualizar opções e realizar reservas.

Sistema (Site): representa a plataforma onde ocorrem as interações entre cliente e administrador.

4.3.3. Casos de Uso

UC10 – Disponibilizar regras de negócio

A empresa franqueadora define as regras que serão aplicadas no sistema, garantindo padronização e controle.

UC01 – Manter o sistema

O administrador gerencia o sistema de forma geral, incluindo funcionalidades essenciais.

Inclui:

UC02 – Cadastrar sorvetes

UC03 – Cadastrar carrinhos

UC02 – Cadastrar sorvetes

Permite ao administrador inserir e gerenciar os produtos disponíveis no sistema.

UC03 – Cadastrar carrinhos

Responsável por registrar os carrinhos disponíveis para locação.

UC04 – Visualizar o site

O cliente acessa o site para visualizar produtos e serviços disponíveis.

UC05 – Enviar dados pessoais

O cliente fornece seus dados para dar continuidade à reserva.

UC06 – Escolher opções disponíveis

O cliente seleciona:

Tipo de sorvete

Carrinho

Data

Horário

UC07 – Realizar pagamento

Etapa em que o cliente efetua o pagamento da reserva.

UC08 – Realizar reserva

Finalização do processo com a confirmação da reserva.

UC09 – Gerenciar reservas

O administrador acompanha e gerencia todas as reservas realizadas.

4.3.4. Relacionamentos Importantes

<<include>> entre:

Manter sistema → Cadastrar sorvetes

Manter sistema → Cadastrar carrinhos

Escolher opções → Realizar pagamento

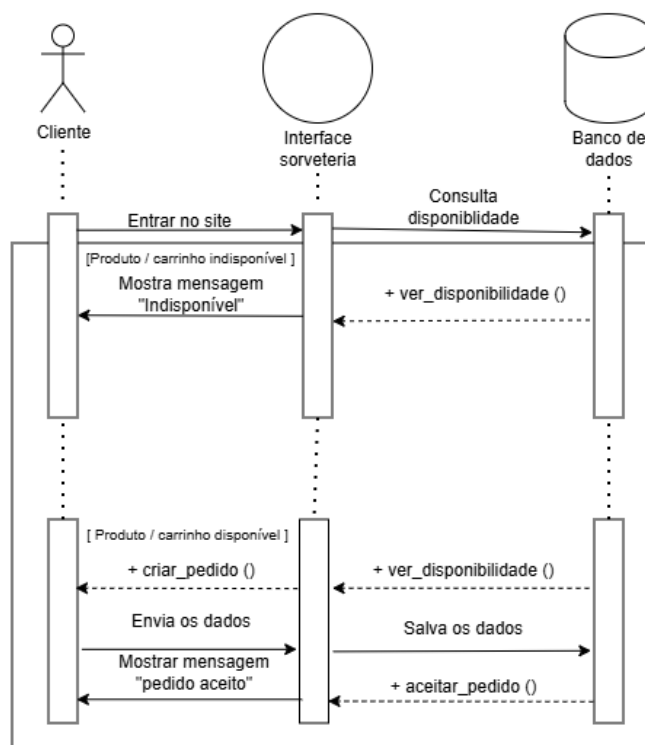
Gerenciar reservas → Realizar reserva

Esses relacionamentos indicam dependência obrigatória entre funcionalidades.

4.3.5. Considerações Finais

O diagrama mostra claramente a separação entre funções administrativas e ações do cliente, além de destacar a dependência entre processos.

4.4. DIAGRAMA DE SEQUÊNCIA



4.4.1. Descrição Geral

De acordo com Guedes (p. 33), o diagrama de sequência é um tipo de diagrama comportamental focado na ordem temporal em que as mensagens são trocadas entre os objetos durante a execução de um processo específico

O diagrama de sequência apresentado descreve o funcionamento do processo de reserva de um carrinho de sorvete dentro do sistema proposto. Ele representa, de forma temporal, como ocorre a troca de mensagens entre o cliente, a interface do site e o banco de dados desde o momento em que o usuário acessa a plataforma até a possível confirmação do pedido.

O fluxo foi modelado considerando dois cenários distintos: um em que há disponibilidade de carrinhos/produtos e outro em que não há, o que impede a continuidade da reserva.

4.4.2. Participantes

Cliente: usuário final que acessa o sistema com a intenção de verificar disponibilidade e realizar uma reserva.

Interface da Sorveteria: camada responsável pela interação com o usuário, recebendo entradas e exibindo respostas, além de se comunicar com o banco de dados.

Banco de Dados: componente responsável por armazenar as informações do sistema, incluindo disponibilidade de carrinhos e registros de pedidos.

4.4.3. Descrição do Fluxo

Acesso inicial e verificação de disponibilidade

O processo tem início quando o cliente acessa o site da sorveteria (“Entrar no site”). A partir dessa ação, a interface realiza automaticamente uma consulta ao banco de dados para verificar a disponibilidade de carrinhos e/ou produtos.

Essa verificação é feita por meio da operação `ver_disponibilidade()`, que retorna à interface se há ou não itens disponíveis para reserva.

Cenário sem disponibilidade

Caso o banco de dados retorne que não há disponibilidade, o sistema interrompe o fluxo nesse ponto. A interface exibe ao cliente a mensagem "Indisponível", informando que não é possível prosseguir com a reserva naquele momento.

Esse cenário representa um fluxo alternativo e é importante para garantir que o sistema trate corretamente situações de indisponibilidade, evitando a criação de pedidos inválidos.

Cenário com disponibilidade (fluxo principal)

Quando há disponibilidade, o cliente pode continuar com o processo normalmente.

Primeiramente, o cliente inicia a ação de criação do pedido (`criar_pedido()`). Em

seguida, são informados os dados necessários para a reserva (como data e itens desejados), que são enviados pela interface ao banco de dados.

O banco de dados, então, realiza o registro dessas informações por meio da operação `aceitar_pedido()`, garantindo que o pedido fique armazenado no sistema.

Após a confirmação do registro, a interface retorna ao cliente uma mensagem de sucesso, indicando que o pedido foi realizado corretamente ("pedido aceito").

4.4.4. Observações sobre o processo

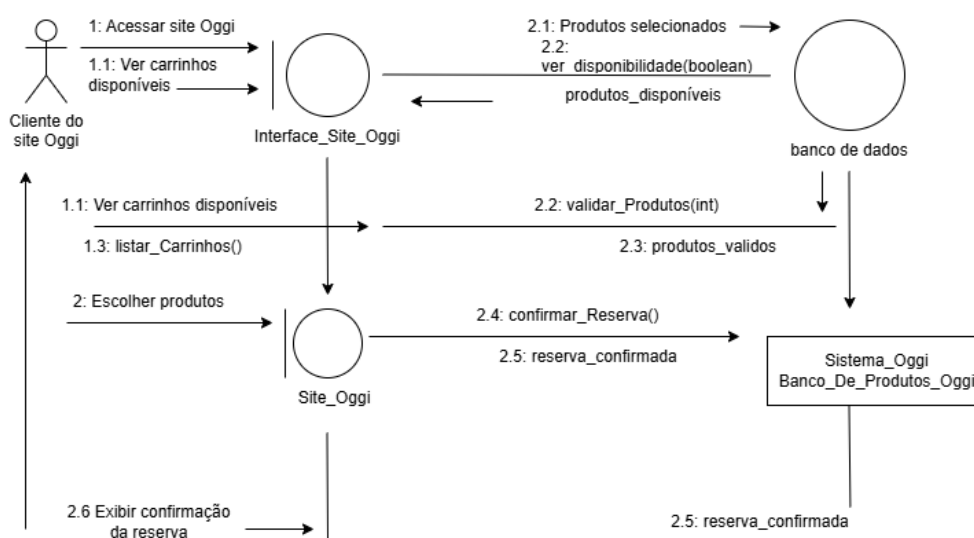
- A verificação de disponibilidade é uma etapa obrigatória antes da criação de qualquer pedido.
- A interface atua como intermediária entre o cliente e o banco de dados.
- O banco de dados não interage diretamente com o cliente.
- O fluxo segue uma sequência linear de eventos.

4.4.5. Considerações finais

O diagrama de sequência permite visualizar de maneira clara como o sistema se comporta durante o processo de reserva, evidenciando tanto o caminho de sucesso quanto o de falha. Essa representação facilita o entendimento da lógica do sistema e contribui para a validação do funcionamento esperado.

Além disso, o modelo reforça a separação de responsabilidades entre os componentes, o que é essencial para a organização e manutenção do sistema ao longo do tempo.

4.5. DIAGRAMA DE COLABORAÇÃO



4.5.1. Descrição Geral

Segundo Guedes (p.35), o diagrama de comunicação, anteriormente denominado diagrama de colaboração até a versão 1.5 da UML, passou a adotar a nomenclatura atual a partir da versão 2.0. Esse diagrama está fortemente associado ao diagrama de sequência, funcionando como seu complemento, uma vez que ambos apresentam informações semelhantes, porém com enfoques distintos. Enquanto o diagrama de sequência enfatiza a ordem temporal das interações, o diagrama de comunicação concentra-se na estrutura dos relacionamentos entre os elementos e nas mensagens trocadas entre eles durante o processo.

O diagrama de colaboração descreve a interação entre os componentes do sistema de reserva de carrinhos de sorvete, destacando como os objetos se comunicam para realizar a funcionalidade de reserva. Diferente do diagrama de sequência, este modelo enfatiza a organização estrutural das interações e o fluxo de mensagens entre os participantes.

O processo representado abrange desde o acesso do cliente ao site até a confirmação da reserva, incluindo a verificação de disponibilidade e validação dos produtos selecionados.

4.5.2. Participantes

Cliente do site Oggi: Usuário que acessa o sistema para visualizar carrinhos disponíveis e realizar uma reserva.

Interface_Site_Oggi: Responsável por intermediar a comunicação entre o cliente e o sistema, recebendo solicitações e exibindo respostas.

Site_Oggi: Representa a camada de aplicação onde ocorre o processamento das ações do usuário, como seleção de produtos e confirmação da reserva.

Banco de Dados: Responsável por armazenar e fornecer informações sobre produtos e disponibilidade.

Sistema_Banco_De_Produtos_Oggi: Componente responsável pela validação dos produtos e confirmação da reserva junto ao banco de dados.

4.5.3. Descrição do Fluxo de Interação

Acesso e consulta inicial

O processo inicia quando o cliente acessa o site (1: Acessar site Oggi). Em seguida, solicita a visualização dos carrinhos disponíveis (1.1).

A interface recebe essa solicitação e encaminha ao sistema a operação de listagem de carrinhos (1.3: listar_Carrinhos()).

Verificação de disponibilidade

A interface realiza uma consulta ao banco de dados para verificar a disponibilidade dos produtos (2.2: ver_disponibilidade(boolean)).

O banco de dados retorna a informação indicando se há produtos disponíveis.

Seleção e validação de produtos

Após visualizar os carrinhos, o cliente escolhe os produtos desejados (2: Escolher produtos).

Essas informações são enviadas ao sistema, que solicita ao banco de dados a validação dos produtos (2.2: validar_Produtos(int)).

O banco retorna à confirmação de produtos válidos (2.3: produtos_validos).

Confirmação da reserva

Com os produtos validados, o sistema executa a operação de confirmação da reserva (2.4: confirmar_Reserva()).

O sistema de banco de produtos processa a solicitação e retorna a confirmação (2.5: reserva_confirmada).

Retorno ao cliente

Por fim, a interface exhibe ao cliente a confirmação da reserva realizada (2.6: Exibir confirmação da reserva), finalizando o fluxo.

4.5.4. Observações sobre o Processo

A numeração das mensagens (1, 1.1, 2.2, etc.) indica a ordem das interações no diagrama.

A interface atua como intermediária entre o cliente e os demais componentes do sistema.

O banco de dados é responsável tanto pela verificação de disponibilidade quanto pela validação dos produtos.

O sistema especializado (Sistema_Banco_De_Produtos_Oggi) centraliza a lógica de confirmação da reserva.

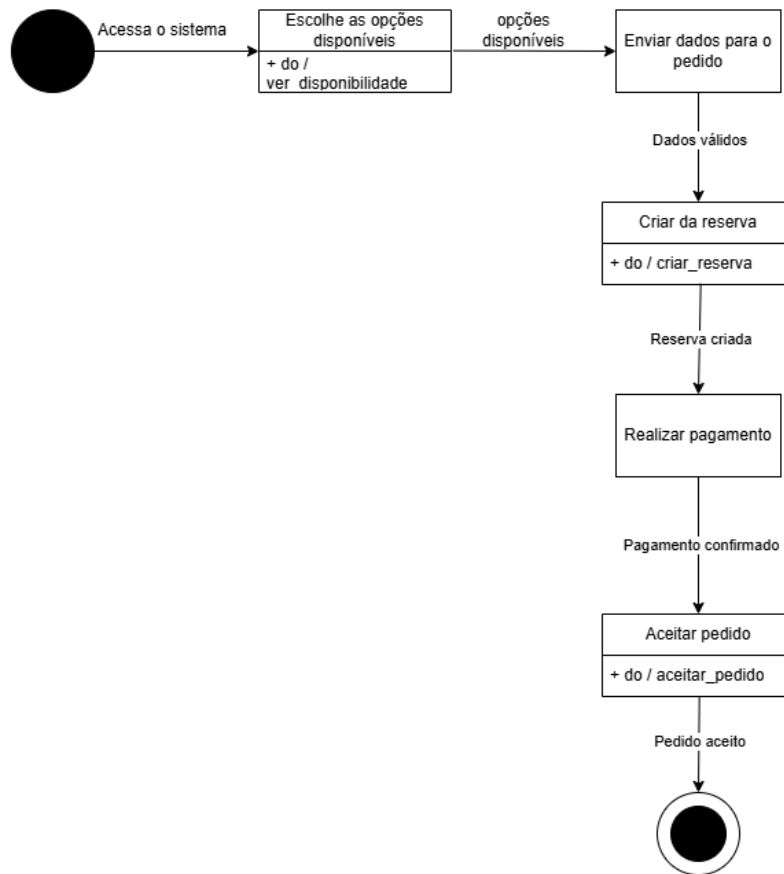
O fluxo segue uma sequência lógica, mas enfatiza as relações entre os objetos em vez do tempo.

4.5.5. Considerações Finais

O diagrama de colaboração permite visualizar de forma clara como os diferentes componentes do sistema interagem para realizar a reserva, destacando a troca de mensagens e as responsabilidades de cada elemento. Essa abordagem facilita o entendimento da arquitetura do sistema e da distribuição das funções entre os objetos.

Além disso, o modelo reforça a importância da comunicação entre interface, sistema e banco de dados, contribuindo para uma implementação organizada, modular e de fácil manutenção.

4.6. DIAGRAMA DE GRÁFICO DE ESTADO



4.6.1. Descrição Geral

Segundo Guedes (p.36), o diagrama de máquina de estados é empregado para demonstrar o comportamento de um elemento do sistema por meio de um conjunto finito de estados e transições. Esse tipo de diagrama pode representar o funcionamento de uma parte específica do sistema, incluindo protocolos de uso ou mudanças de estado ao longo do processo.

O diagrama de estado descreve o ciclo de vida de uma reserva dentro do sistema de carrinhos de sorvete, evidenciando as transições entre os diferentes estados desde o acesso inicial até a finalização do pedido. Ele representa de forma dinâmica como o sistema responde às ações do usuário e às validações internas.

O fluxo mostra as etapas sequenciais necessárias para que uma reserva seja realizada com sucesso, incluindo verificação de disponibilidade, criação do pedido, pagamento e confirmação final.

4.6.2. Estados Participantes

Estado inicial: Representa o ponto de entrada do sistema, quando o usuário acessa a plataforma.

Escolher opções disponíveis: Estado em que o usuário visualiza e seleciona os carrinhos ou produtos disponíveis, incluindo a ação de verificação de disponibilidade.

Enviar dados para o pedido: Estado em que o usuário fornece as informações necessárias para realizar a reserva, como itens e dados do evento.

Criar reserva: Estado responsável pela criação do pedido no sistema, após a validação dos dados informados.

Realizar pagamento: Estado em que o usuário efetua o pagamento referente à reserva.

Aceitar pedido: Estado em que o sistema confirma o pedido após a validação do pagamento.

Estado final: Representa o encerramento do processo com o pedido aceito e concluído.

4.6.3. Descrição do Fluxo de Estados

Início do processo

O fluxo inicia no estado inicial, quando o usuário acessa o sistema (“Acessa o sistema”). Em seguida, ocorre a transição para o estado de escolha de opções disponíveis.

Seleção e envio de dados

Após visualizar as opções, o usuário seleciona os produtos disponíveis e avança para o estado de envio de dados do pedido.

Caso os dados sejam válidos, o sistema permite a transição para a criação da reserva.

Criação da reserva

No estado de criação da reserva, o sistema executa a ação de registrar o pedido (criar_reserva). Após isso, a reserva é considerada criada, permitindo avançar para a próxima etapa.

Processamento do pagamento

Com a reserva criada, o usuário realiza o pagamento. Quando o pagamento é confirmado, ocorre a transição para o estado de aceitação do pedido.

Finalização do pedido

No estado de aceitar pedido, o sistema executa a ação de confirmação (aceitar_pedido). Após essa etapa, o pedido é considerado aceito e o fluxo é encerrado no estado final.

4.6.4. Observações sobre o Processo

O fluxo segue uma sequência linear de estados bem definidos.

Cada transição depende de uma condição, como validação de dados ou confirmação de

pagamento.

O processo só é concluído após a aceitação do pedido.

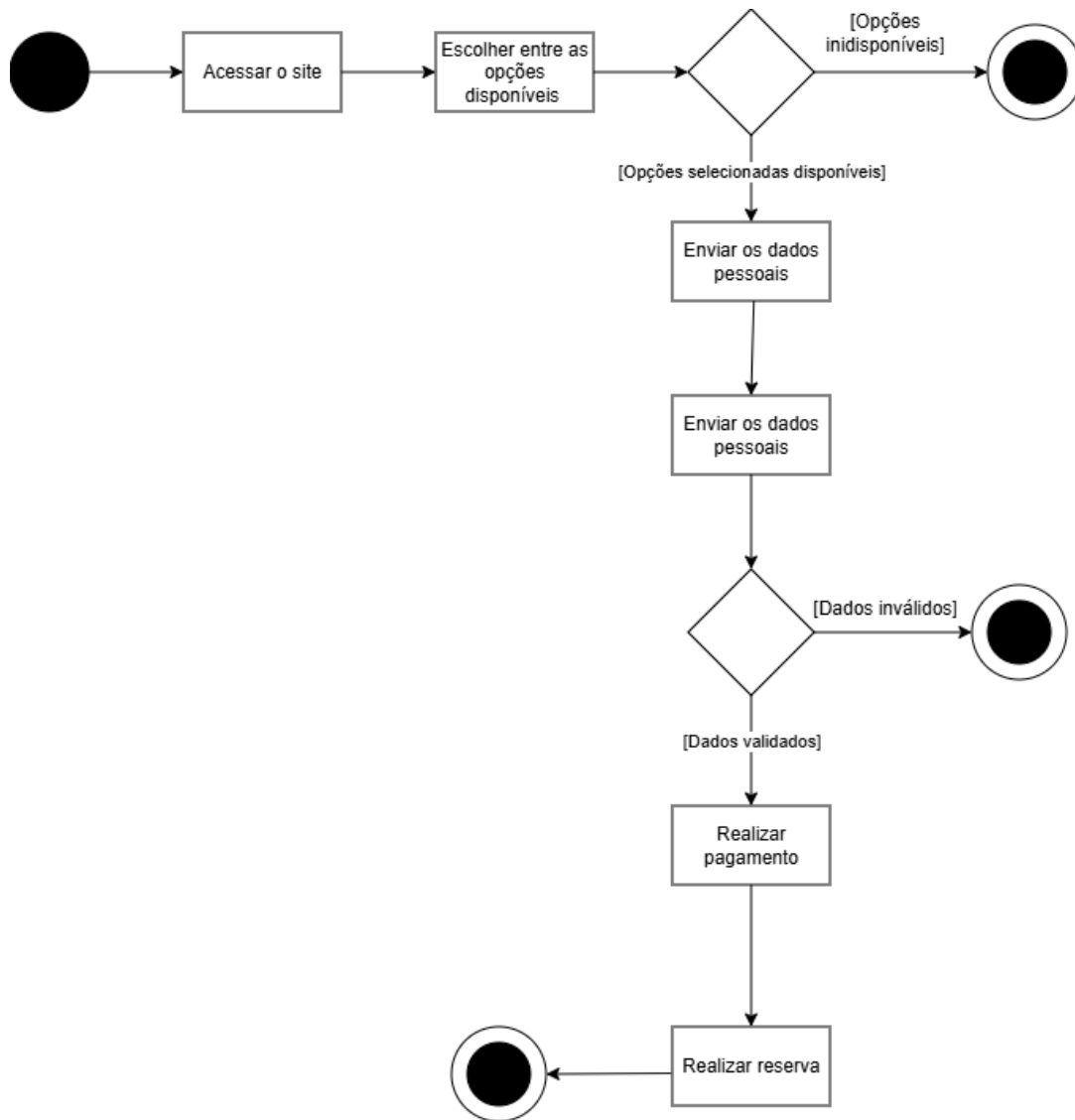
O diagrama não apresenta estados alternativos (como falha de pagamento), focando apenas no fluxo principal.

4.6.5. Considerações Finais

O diagrama permite visualizar claramente o ciclo de vida de uma reserva dentro do sistema, evidenciando as etapas necessárias para sua conclusão. Essa representação auxilia na compreensão das mudanças de estado do sistema e garante que todas as transições estejam corretamente definidas.

Além disso, o modelo contribui para a implementação de regras de negócio mais consistentes, assegurando que cada etapa seja executada na ordem correta e que o sistema responda adequadamente às ações do usuário.

4.7. DIAGRAMA DE ATIVIDADES



4.7.1. Descrição Geral

Guedes (p.36) destaca que o diagrama de atividade descreve os passos necessários para a realização de uma atividade específica, podendo representar desde métodos mais complexos até algoritmos ou processos completos.

O diagrama de atividades representa o fluxo completo do processo de reserva de carrinhos de sorvete dentro do sistema. Ele descreve, de forma sequencial, as ações realizadas pelo cliente e as decisões tomadas pelo sistema desde o acesso inicial até a finalização da reserva.

O modelo contempla tanto o fluxo principal (quando tudo ocorre corretamente) quanto os fluxos alternativos, como indisponibilidade de opções e invalidação de dados.

4.7.2. Descrição do Processo

O processo tem início quando o cliente acessa o site da plataforma. A partir desse

momento, o usuário passa a interagir com o sistema com o objetivo de realizar uma reserva.

Após o acesso, o cliente seleciona entre as opções disponíveis, como tipo de sorvete, carrinho, data e horário desejados. Essa etapa é fundamental, pois define os parâmetros da reserva.

Em seguida, o sistema realiza uma verificação para identificar se as opções escolhidas estão disponíveis. Essa verificação é essencial para garantir que não haja conflitos, como reservas duplicadas ou indisponibilidade de recursos.

Caso o sistema identifique que as opções selecionadas não estão disponíveis, o processo é encerrado nesse ponto. Isso impede que o cliente prossiga com uma reserva inválida, garantindo a integridade das informações.

Por outro lado, se as opções estiverem disponíveis, o processo continua normalmente. O cliente então deve fornecer seus dados pessoais, que são necessários para a identificação e registro da reserva.

Após o envio dessas informações, o sistema realiza uma validação dos dados fornecidos. Essa etapa tem como objetivo garantir que os dados estejam corretos e completos.

Se os dados forem considerados inválidos, o processo é novamente interrompido, impedindo que a reserva seja realizada com informações incorretas.

Caso os dados sejam validados com sucesso, o cliente prossegue para a etapa de pagamento. Nessa fase, o sistema processa a transação financeira necessária para confirmar a reserva.

Com o pagamento realizado, o sistema finaliza o processo registrando a reserva como concluída. Nesse momento, a reserva é efetivada e o fluxo chega ao seu fim.

4.7.3. Considerações sobre o Fluxo

O processo segue uma sequência lógica e estruturada, garantindo que todas as etapas necessárias sejam cumpridas antes da conclusão da reserva.

Destacam-se dois pontos de decisão importantes:

A verificação de disponibilidade das opções escolhidas

A validação dos dados pessoais do cliente

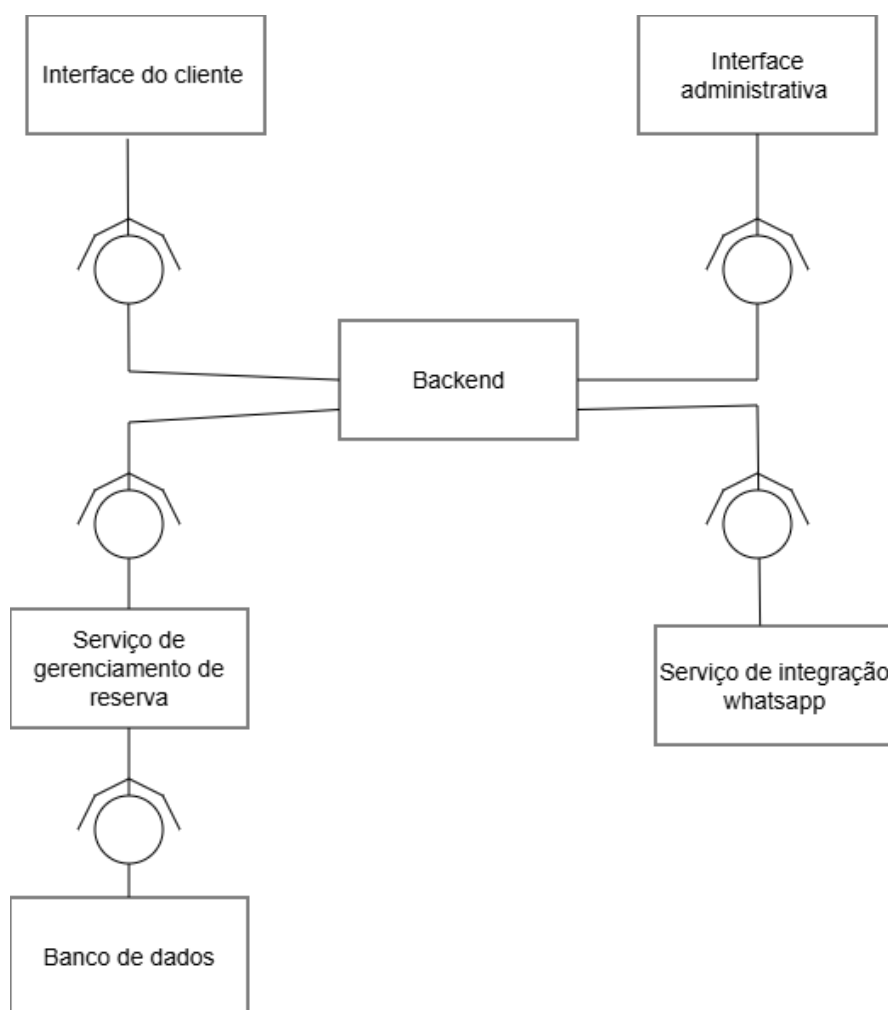
Essas verificações são essenciais para evitar inconsistências no sistema e garantir a confiabilidade das reservas.

4.7.4. Considerações Finais

O diagrama de atividades permite compreender claramente o comportamento do sistema durante o processo de reserva, evidenciando tanto o fluxo principal quanto as possíveis interrupções.

Essa representação contribui para a validação das regras de negócio e auxilia no desenvolvimento do sistema, garantindo que todas as etapas sejam executadas de forma correta e segura.

4.8. DIAGRAMA DE COMPONENTES



4.8.1. Descrição Geral

Guedes (p.39) explica que o diagrama de componentes está fortemente relacionado à implementação e à arquitetura do sistema. Ele descreve como o sistema será estruturado em termos de módulos de código, bibliotecas, formulários, arquivos e demais partes executáveis. Esse diagrama auxilia no entendimento de como os componentes interagem entre si e como contribuem para o funcionamento adequado do software.

O diagrama de componentes descreve a arquitetura do sistema de reserva de carrinhos de sorvete, evidenciando como os diferentes módulos estão organizados e como se comunicam

entre si. Ele representa uma visão estrutural de alto nível, destacando a separação entre interfaces, serviços e backend.

O modelo demonstra como o sistema é dividido em componentes independentes, permitindo melhor organização, manutenção e escalabilidade da aplicação.

4.8.2. Componentes Participantes

Interface do cliente:

Responsável pela interação direta com o usuário final, permitindo acessar o sistema, visualizar opções e realizar reservas.

Interface administrativa:

Utilizada pelos administradores do sistema para gerenciar reservas, produtos e demais informações.

Backend:

Componente central do sistema, responsável por processar as regras de negócio, receber requisições das interfaces e coordenar a comunicação com os demais serviços.

Serviço de gerenciamento de reserva:

Responsável por toda a lógica relacionada às reservas, como criação, alteração, validação e cancelamento.

Serviço de integração WhatsApp:

Permite a comunicação externa com o cliente, enviando notificações e confirmações de reservas através do WhatsApp.

Banco de dados:

Responsável pelo armazenamento persistente das informações do sistema, como clientes, reservas, carrinhos e produtos.

4.8.3. Descrição das Interações

Comunicação com o Backend

As interfaces (cliente e administrativa) se comunicam diretamente com o backend, enviando requisições e recebendo respostas. O backend atua como ponto central de controle.

Processamento de reservas

O backend encaminha as operações relacionadas às reservas para o serviço de gerenciamento de reserva, que executa toda a lógica necessária para garantir o correto funcionamento do processo.

Persistência de dados

O serviço de gerenciamento de reserva se comunica com o banco de dados para armazenar e recuperar informações, garantindo a persistência dos dados.

Integração externa

O backend também se conecta ao serviço de integração com WhatsApp, responsável por enviar mensagens de confirmação e notificações aos usuários.

4.8.4. Observações sobre a Arquitetura

O backend é o núcleo do sistema, centralizando as regras de negócio.

As interfaces são desacopladas da lógica interna, facilitando mudanças na camada de apresentação.

Os serviços são especializados, cada um com uma responsabilidade específica.

O uso de integração externa (WhatsApp) amplia a comunicação com o cliente.

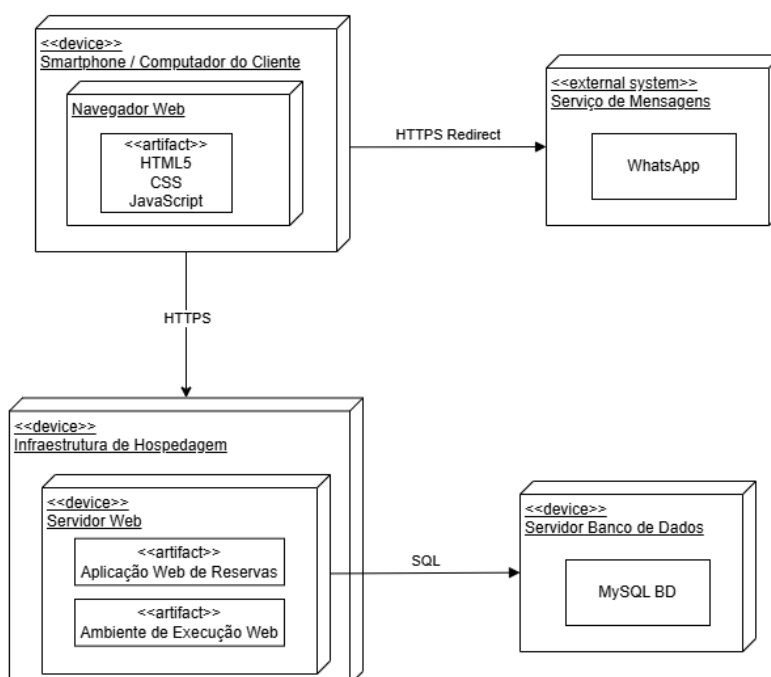
A separação em componentes favorece a escalabilidade e manutenção do sistema.

4.8.5. Considerações Finais

O diagrama de componentes permite compreender a arquitetura do sistema de forma clara e organizada, evidenciando como os diferentes módulos interagem para garantir o funcionamento da aplicação. Essa abordagem facilita o desenvolvimento, pois promove a separação de responsabilidades e possibilita a evolução independente de cada componente.

Além disso, o modelo reforça boas práticas de arquitetura, como modularização e baixo acoplamento, contribuindo para um sistema mais robusto, flexível e de fácil manutenção.

4.9. DIAGRAMA DE IMPLANTAÇÃO



4.9.1. Descrição Geral

Segundo Guedes (p.39), o diagrama de implantação é utilizado para representar a estrutura física do sistema, incluindo os requisitos de hardware, como servidores, estações de trabalho, topologias de rede e protocolos de comunicação. Além disso, esse diagrama demonstra como os componentes do sistema são distribuídos em diferentes ambientes físicos, especialmente em cenários onde a aplicação é executada em múltiplos servidores.

O diagrama de implantação descreve a infraestrutura física e a distribuição dos componentes do sistema de reserva de carrinhos de sorvete. Ele evidencia onde cada parte do sistema está hospedada, como os dispositivos se conectam e quais tecnologias estão envolvidas na execução da aplicação.

Esse tipo de diagrama fornece uma visão de alto nível da arquitetura, destacando servidores, dispositivos, comunicação entre nós e a integração com sistemas externos.

4.9.2. Nós (Dispositivos) Participantes

Smartphone / Computador do Cliente: Dispositivo utilizado pelo usuário para acessar o sistema. Nele está presente um navegador web que executa a interface da aplicação utilizando tecnologias como HTML5, CSS e JavaScript.

Navegador Web: Ambiente onde a interface do sistema é renderizada, permitindo a interação do usuário com a aplicação.

Infraestrutura de Hospedagem: Ambiente onde o sistema está implantado, responsável por hospedar o servidor web e garantir sua disponibilidade.

Servidor Web: Responsável por executar a aplicação web de reservas. Contém o ambiente de execução e os artefatos da aplicação.

Servidor Banco de Dados: Responsável pelo armazenamento dos dados do sistema, utilizando um banco de dados MySQL.

Serviço de Mensagens (WhatsApp): Sistema externo utilizado para envio de mensagens e notificações aos usuários.

4.9.3. Artefatos do Sistema

Aplicação Web de Reservas: responsável pela lógica do sistema e processamento das requisições.

Ambiente de Execução Web: plataforma onde a aplicação é executada no servidor.

Frontend (HTML5, CSS, JavaScript): executado no navegador do cliente.

Banco de Dados MySQL: responsável pela persistência das informações.

4.9.4. Comunicação entre os Componentes

Cliente → Servidor Web (HTTPS):

O acesso do usuário ao sistema ocorre via navegador, utilizando protocolo seguro HTTPS.

Servidor Web → Banco de Dados (SQL):

O servidor se comunica com o banco de dados para realizar operações de leitura e escrita.

Cliente → Serviço de Mensagens (HTTPS Redirect):

O sistema pode redirecionar o usuário ou enviar informações para o serviço de mensagens (WhatsApp).

4.9.5. Observações sobre a Arquitetura

A arquitetura segue o modelo cliente-servidor.

O uso de HTTPS garante maior segurança na comunicação.

A separação entre servidor web e banco de dados melhora a escalabilidade e segurança.

A integração com um sistema externo (WhatsApp) amplia as funcionalidades do sistema.

O frontend é executado no cliente, enquanto o backend está centralizado no servidor.

4.9.6. Considerações Finais

O diagrama de implantação permite compreender como o sistema está distribuído fisicamente e como seus componentes são executados em diferentes ambientes. Essa visão é fundamental para planejamento de infraestrutura, segurança e desempenho.

Além disso, o modelo evidencia boas práticas como separação de camadas, uso de protocolos seguros e integração com serviços externos, contribuindo para um sistema mais robusto, escalável e eficiente.

5. ESCOPO

O sistema proposto contempla módulos integrados para apoiar a operação de uma empresa de locação de carrinhos de sorvete para eventos, abrangendo controle administrativo.

No módulo de gestão de clientes, o sistema permitirá o cadastro de informações básicas dos clientes, como nome, telefone e e-mail, bem como o registro do endereço onde o evento será realizado. Também será mantido um histórico de eventos anteriores, possibilitando melhor acompanhamento e relacionamento com os clientes. Adicionalmente, o sistema deverá realizar comunicação automática com o cliente, enviando confirmações e notificações por meio de WhatsApp.

A gestão de reservas permitirá que o cliente selecione a data e o horário do evento, enquanto o sistema realizará a validação automática da disponibilidade do carrinho. Será possível aplicar regras de antecedência mínima para reservas garantindo maior segurança no processo de agendamento.

No módulo de gestão de produtos, o sistema possibilitará o cadastro dos sabores de sorvete disponíveis, bem como o controle de estoque por sabor. O cliente poderá escolher a quantidade desejada para cada evento.

O sistema contará com diferentes modelos de serviço, permitindo ao cliente escolher entre a opção de somente locação do carrinho, carrinho acompanhado de produtos, além disso, será responsável pela emissão de comprovantes, garantindo maior organização financeira, oferecendo também a opção de retirada do carrinho pelo próprio cliente. O sistema permitirá o controle do horário de entrega e retirada, do tempo máximo de uso do equipamento e do registro de devolução do carrinho.

As regras de negócio incluirão políticas automáticas de cancelamento, cálculo de multas em casos de cancelamento fora do prazo, definição de quantidade mínima de produtos para liberação do carrinho gratuito, e controle de estoque para evitar overbooking.

6. BANCO DE DADOS

6.1. MODELO CONCEITUAL

6.1.1. minimundo

O sistema foi desenvolvido para gerenciar o aluguel de carrinhos de sorvete para eventos, permitindo que clientes realizem reservas escolhendo uma data e os produtos desejados.

Os clientes podem realizar reservas informando seus dados pessoais, como nome, telefone, endereço e e-mail. Cada reserva registra a data do evento, a disponibilidade do carrinho, o valor total do pedido, o status da solicitação e uma breve descrição do evento.

Cada reserva está associada a um carrinho de sorvete, que possui informações como preço da diária e status de disponibilidade. Isso permite ao sistema controlar quais carrinhos estão disponíveis para aluguel em determinadas datas.

Além disso, durante a criação da reserva, o cliente pode selecionar diferentes produtos (sabores de sorvete). Como uma reserva pode conter vários produtos e um produto pode aparecer em várias reservas, essa relação é do tipo muitos para muitos (N:N). Para resolver esse tipo de relacionamento, é criada uma entidade intermediária chamada *Reserva_produto*, responsável por registrar quais produtos pertencem a cada reserva.

Dessa forma, o sistema consegue controlar clientes, reservas, carrinhos disponíveis e os produtos selecionados para cada evento.

6.1.2. MODELO ENTIDADE RELACIONAMENTO (MER)

6.1.3. ENTIDADES

Cliente: A entidade Cliente existe para armazenar as informações das pessoas que realizam reservas de carrinhos de sorvete para eventos. Ela é essencial para identificar quem solicitou determinado pedido e manter o histórico de reservas realizadas. Essa entidade contém informações como nome, telefone, endereço e e-mail, permitindo que a empresa entre em contato com o cliente e organize os pedidos de forma adequada.

Reserva: A entidade Reserva tem como objetivo registrar as solicitações feitas pelos clientes para aluguel do carrinho de sorvete. Cada reserva armazena informações importantes sobre o evento, como data, disponibilidade, valor total, status da solicitação e uma descrição. Além disso, ela estabelece a ligação entre o cliente, o carrinho que será utilizado e os produtos

escolhidos para o evento.

Carrinho: A entidade Carrinho representa os carrinhos de sorvete disponíveis para aluguel. Ela é importante para controlar os equipamentos disponíveis para os eventos. Cada carrinho possui informações como preço da diária e status, indicando se está disponível, reservado ou indisponível.

Produto: A entidade Produto representa os sabores de sorvete disponíveis no sistema. Ela permite que o cliente escolha quais produtos deseja incluir na reserva. Cada produto possui informações como nome do produto, preço e quantidade disponível.

Reserva_produto: A entidade Reserva_produto existe para resolver o relacionamento do tipo muitos para muitos (N:N) entre Reserva e Produto. Como uma reserva pode conter vários produtos e um mesmo produto pode fazer parte de diversas reservas, essa tabela intermediária é necessária para registrar essa associação. Ela armazena as chaves estrangeiras das duas entidades e permite que o sistema saiba quais produtos foram selecionados em cada reserva.

6.1.4. ATRIBUTOS

Cliente:

- id_cliente (PK) – Identificador único do cliente no sistema;
- nome – Nome completo do cliente;
- telefone – Número de telefone para contato;
- endereco – Endereço onde o evento será realizado;
- email – E-mail do cliente.

Reserva:

- id_reserva (PK) – Identificador único da reserva;
- id_cliente (FK) – Cliente responsável pela reserva;
- id_carrinho (FK) – Carrinho que será utilizado no evento;
- data_evento – Data em que o evento ocorrerá;
- disponibilidade – Indica se o carrinho está disponível para a data escolhida;
- valor_total – Valor total da reserva;
- status – Situação da reserva (pendente, aceita ou negada);
- descricao – Observações ou descrição do evento.

Carrinho:

- id_carrinho (PK) – Identificador único do carrinho;
- preco_diaria – Valor cobrado pela diária do carrinho;
- status – Indica a condição do carrinho (se está quebrado, funcionando).

Produto:

- id_produto (PK) – Identificador único do produto;
- nome_produto – Nome do sabor de sorvete;

- preco – Valor do produto;
- quantidade – Quantidade disponível em estoque.

Reserva_produto:

- id_reserva_produto (PK) – Identificador único do relacionamento;
- id_reserva (FK) – Reserva associada;
- id_produto (FK) – Produto selecionado na reserva.

6.1.5. RELACIONAMENTOS

Cliente → Realiza → Reserva (1:N)

Um cliente pode realizar várias reservas, mas cada reserva pertence a apenas um cliente.

Reserva → Utiliza → Carrinho (N:1)

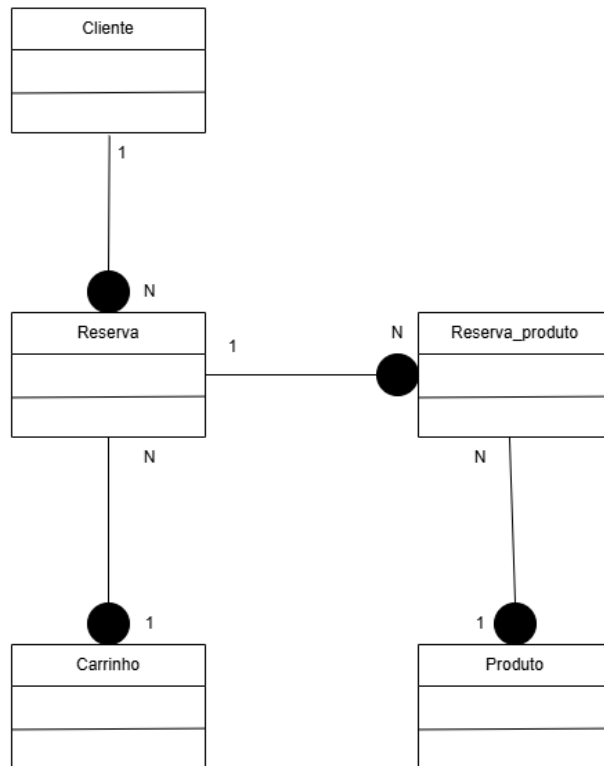
Uma reserva utiliza um único carrinho, enquanto um carrinho pode ser utilizado em várias reservas em diferentes datas.

Reserva → Contém → Produto (N:N)

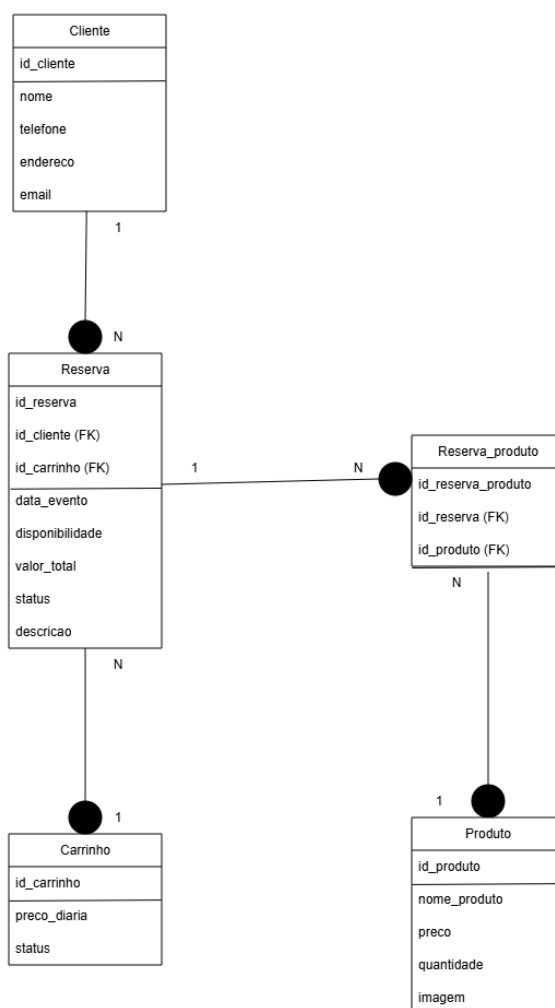
Uma reserva pode conter vários produtos e um mesmo produto pode aparecer em várias reservas.

Para resolver esse relacionamento N:N, foi criada a entidade Reserva_produto, que funciona como uma tabela intermediária ligando as duas entidades e armazenando suas chaves estrangeiras.

6.1.6. DER (Diagrama Entidade Relacionamento)



6.2. MODELO LÓGICO



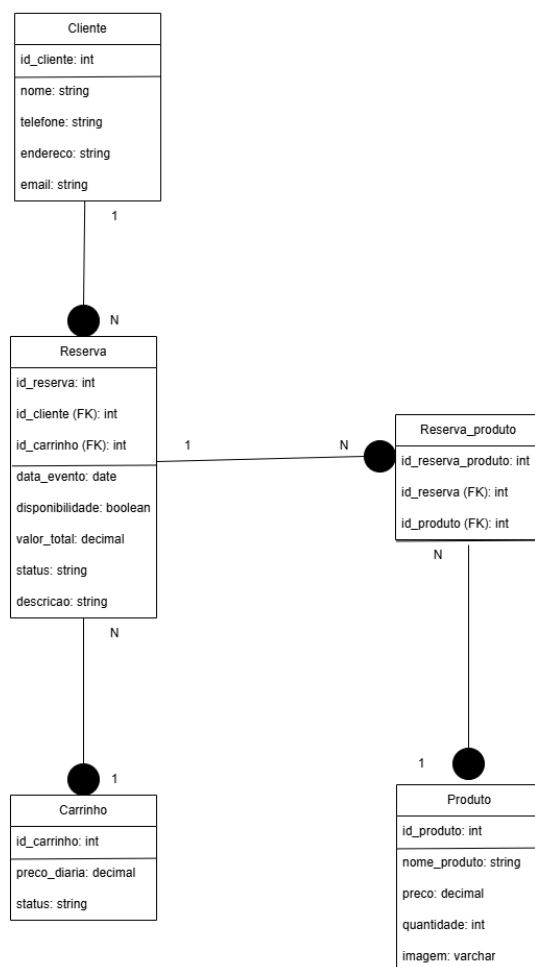
O modelo lógico representa a evolução do modelo conceitual, trazendo maior detalhamento das estruturas que comporão o banco de dados. Nessa etapa, as entidades passam a ser transformadas em tabelas, os atributos são definidos com mais precisão e os relacionamentos são convertidos em chaves primárias e estrangeiras. Apesar desse nível maior de detalhamento, ainda não são consideradas as particularidades de um sistema gerenciador de banco de dados específico.

Segundo Machado (2014, p.20), o modelo lógico descreve as estruturas do banco de dados de acordo com a abordagem adotada, garantindo coerência e integridade. No modelo apresentado, observa-se a definição clara das tabelas como Cliente, Reserva, Carrinho, Produto e Reserva_Produto, incluindo seus atributos e tipos de dados de forma mais organizada.

Outro ponto importante é a implementação das chaves estrangeiras, que garantem a integridade referencial entre as tabelas. Por exemplo, a tabela Reserva possui uma chave estrangeira que referencia o cliente, assegurando que toda reserva esteja vinculada a um cliente válido. Da mesma forma, a tabela associativa Reserva_Produto permite representar corretamente a relação muitos-para-muitos entre reservas e produtos.

Portanto, o modelo lógico é essencial para estruturar o banco de dados de forma consistente, servindo como um guia detalhado para a implementação física. Ele garante que as regras definidas no modelo conceitual sejam mantidas, preparando o sistema para a próxima etapa.

6.3. MODELO FÍSICO



O modelo físico é a etapa final do projeto de banco de dados, onde toda a estrutura planejada é efetivamente implementada em um sistema gerenciador de banco de dados (SGBD). Nessa fase, são utilizados comandos específicos, como SQL, para criar tabelas, definir tipos de dados reais, índices, restrições e demais recursos necessários para o funcionamento do sistema.

De acordo com Machado (2014, p.21), essa fase consiste na montagem do banco de dados no dicionário de dados do sistema, transformando o projeto teórico em um sistema real e operacional. No modelo físico apresentado, é possível observar a definição detalhada dos tipos de dados, como inteiros, strings, decimais e booleanos, além da implementação das chaves primárias e estrangeiras.

Outro aspecto relevante do modelo físico é a preocupação com desempenho e armazenamento. Nessa etapa, podem ser definidos índices para otimizar consultas, restrições para garantir a integridade dos dados e ajustes específicos do SGBD utilizado. Isso torna o banco de dados mais eficiente e seguro para uso em ambiente real.

Assim, o modelo físico concretiza todo o planejamento realizado nas etapas anteriores, garantindo que o sistema seja funcional, confiável e eficiente. Ele representa a materialização do banco de dados, permitindo o armazenamento e a manipulação das informações de forma prática no sistema de reservas.

7. TECNOLOGIAS UTILIZADAS

7.1. ORM (OBJECT-RELATIONAL MAPPING)

O ORM (Object-Relational Mapping) é uma técnica de programação que permite converter dados entre sistemas orientados a objetos e bancos de dados relacionais. Ele funciona como uma camada intermediária que traduz objetos do código da aplicação em registros de tabelas do banco de dados, e vice-versa. Isso permite que os desenvolvedores trabalhem com objetos da linguagem de programação em vez de escrever diretamente comandos SQL (FOWLER, 2006).

Com o uso do ORM, cada tabela do banco de dados geralmente corresponde a uma classe no sistema, enquanto cada linha da tabela representa um objeto dessa classe. As colunas da tabela são mapeadas para atributos do objeto. Dessa forma, operações como inserir, atualizar, consultar ou excluir dados podem ser feitas por meio de métodos e objetos da linguagem de programação (DJANGO SOFTWARE FOUNDATION, 2026).

No desenvolvimento do sistema de reservas do carrinho de sorvete, será utilizado o conceito de ORM para realizar a integração entre a aplicação e o banco de dados. Esse padrão permite que as informações do sistema sejam representadas por meio de objetos no código, enquanto os dados são armazenados em tabelas no banco de dados relacional. Dessa forma, cada entidade importante do sistema, como reservas, sabores e carrinhos disponíveis, poderá ser representada por classes dentro da aplicação.

Para o sistema proposto, o ORM será utilizado para estruturar o relacionamento entre as principais entidades do projeto, como reservas, clientes, carrinhos e sabores, garantindo que as informações possam ser armazenadas, consultadas e atualizadas de maneira organizada no banco de dados. A definição detalhada da tecnologia e da implementação prática será realizada nas etapas posteriores do desenvolvimento.

7.2.MVC (MODEL-VIEW-CONTROLLER)

O MVC (Model–View–Controller) é um padrão de arquitetura de software utilizado para organizar e estruturar aplicações, separando responsabilidades em três componentes principais: Model, View e Controller. Essa separação facilita o desenvolvimento, manutenção e escalabilidade do sistema, pois cada parte possui uma função específica dentro da aplicação. O objetivo do MVC é reduzir o acoplamento entre a interface do usuário e a lógica de negócio (REENSKAUG, 1979).

O Model é responsável por representar os dados da aplicação e as regras de negócio. Ele gerencia a lógica relacionada aos dados, como validações, cálculos e comunicação com o banco de dados. Em outras palavras, o Model é a parte que define como os dados são armazenados, manipulados e processados, independentemente de como eles serão exibidos ao usuário.

A View é a camada responsável pela interface com o usuário, ou seja, pela apresentação dos dados. Ela exibe as informações vindas do Model de forma visual, como páginas web, telas ou relatórios. Já o Controller atua como intermediário entre o Model e a View, recebendo as ações do usuário, processando essas solicitações e atualizando o Model ou a View conforme necessário.

No sistema da sorveteria desenvolvido para o projeto, o Model representa as entidades do sistema, como produtos (sorvetes, sabores e adicionais), clientes, pedidos e vendas. Essa camada é responsável por armazenar e manipular as informações no banco de dados, garantindo que os dados cadastrados estejam corretos e disponíveis para o restante da aplicação.

A View corresponde às telas que o usuário utiliza para interagir com o sistema, como as interfaces de cadastro de produtos, registro de pedidos e visualização das vendas. No caso do sistema da sorveteria, essas telas permitem que o atendente selecione os sabores, registre pedidos dos clientes e visualize informações de forma clara e organizada.

Já o Controller atua como intermediário entre o Model e a View. Quando o usuário realiza alguma ação, como cadastrar um novo sabor de sorvete ou registrar uma venda, o Controller recebe essa solicitação, processa as informações e aciona o Model para salvar ou buscar os dados necessários. Depois disso, o Controller atualiza a View com as informações corretas. Dessa forma, o sistema da sorveteria mantém uma estrutura organizada, facilitando futuras melhorias, manutenção do código e inclusão de novas funcionalidades.

7.3.DJANGO

7.3.1. MVR DO DJANGO

A arquitetura MVT (Model-View-Template) é o padrão utilizado pelo framework

Django para organizar o desenvolvimento de aplicações web. Esse modelo tem como objetivo separar as responsabilidades do sistema em diferentes camadas, facilitando a manutenção, organização e escalabilidade do projeto. Cada componente possui uma função específica, o que contribui para um código mais limpo e estruturado (DJANGO SOFTWARE FOUNDATION, 2026).

O componente Model é responsável pela definição da estrutura dos dados e pela lógica de negócio relacionada ao banco de dados. Nele são declaradas as classes que representam as entidades do sistema, como clientes, reservas e produtos. Além disso, o Model utiliza o ORM do Django para realizar operações no banco de dados de forma automatizada, sem a necessidade de escrever comandos SQL diretamente.

O componente View atua como intermediário entre o Model e o Template. Ele é responsável por processar as requisições do usuário, acessar os dados necessários no Model e retornar uma resposta adequada. As Views podem conter regras de negócio, validações e lógica de controle, sendo fundamentais para o funcionamento da aplicação.

O Template é a camada responsável pela interface visual do sistema. Ele define como os dados serão apresentados ao usuário, utilizando HTML combinado com a linguagem de template do Django. Essa separação permite que a parte visual seja alterada sem interferir na lógica do sistema, garantindo maior flexibilidade no desenvolvimento.

A interação entre esses três componentes ocorre de forma integrada: o usuário faz uma requisição, a View processa essa requisição, consulta o Model quando necessário e retorna os dados para o Template, que os exibe na interface. Dessa forma, o padrão MVT proporciona uma estrutura organizada, eficiente e adequada para o desenvolvimento de aplicações web robustas.

7.3.2. ORM DO DJANGO

O ORM (Object-Relational Mapping) do Django é uma ferramenta que permite a interação com o banco de dados utilizando programação orientada a objetos. Em vez de escrever comandos SQL diretamente, o desenvolvedor manipula classes e objetos em Python, tornando o desenvolvimento mais simples, intuitivo e produtivo (DJANGO SOFTWARE FOUNDATION, 2026).

No Django, o ORM é implementado por meio das classes definidas no Model. Cada classe representa uma tabela no banco de dados, enquanto os atributos da classe correspondem às colunas dessa tabela. Dessa forma, ao criar um objeto em Python, o desenvolvedor está, na prática, criando um registro no banco de dados.

Uma das principais vantagens do ORM é a abstração do banco de dados. O desenvolvedor não precisa se preocupar com diferenças entre sistemas como MySQL, PostgreSQL ou SQLite, pois o Django se encarrega de traduzir as operações realizadas em código Python para comandos SQL compatíveis com o banco utilizado.

Além disso, o ORM facilita operações comuns como criação, leitura, atualização e exclusão de dados (CRUD). Por meio de métodos simples, é possível realizar consultas, aplicar filtros e manipular dados com poucas linhas de código. Isso reduz a complexidade e diminui a chance de erros, especialmente para desenvolvedores iniciantes.

Em resumo, o ORM do Django simplifica significativamente o acesso ao banco de dados, tornando o código mais legível e organizado. Ele permite que o desenvolvedor foque na lógica da aplicação, enquanto o framework gerencia a comunicação com o banco de dados, garantindo eficiência e produtividade no desenvolvimento.

7.4. LGPD

A proteção dos dados pessoais dos usuários é um aspecto fundamental no desenvolvimento do sistema, especialmente considerando as diretrizes estabelecidas pela Lei Geral de Proteção de Dados Pessoais (LGPD – Lei nº 13.709/2018). O sistema foi projetado com o objetivo de garantir a segurança, privacidade e integridade das informações coletadas, adotando boas práticas de segurança da informação e tratamento de dados (BRASIL, 2018).

Os dados coletados pelo sistema, como nome, telefone, endereço e e-mail, são utilizados exclusivamente para a realização e gerenciamento das reservas solicitadas pelos usuários. Não há compartilhamento dessas informações com terceiros sem o devido consentimento do titular, assegurando que o tratamento dos dados esteja alinhado com os princípios da finalidade e necessidade previstos na LGPD.

Para garantir a segurança das informações, o sistema adota mecanismos de proteção como validação de dados de entrada, controle de acesso por níveis de usuário (administrador e cliente) e restrição de acesso às informações sensíveis. Além disso, recomenda-se a utilização de conexões seguras (HTTPS) e armazenamento protegido no banco de dados, reduzindo riscos de acesso não autorizado.

O acesso aos dados é limitado apenas a usuários autorizados, sendo o administrador responsável pelo gerenciamento das informações e das reservas. Também são implementadas medidas para evitar acessos indevidos, como autenticação de usuários e proteção das rotas do sistema, garantindo que apenas pessoas autorizadas possam visualizar ou modificar os dados.

Por fim, o sistema prevê a transparência no uso das informações, permitindo que o

usuário tenha ciência sobre quais dados estão sendo coletados e sua finalidade. Dessa forma, o projeto busca estar em conformidade com a LGPD, promovendo a segurança da informação e o respeito à privacidade dos usuários.

8. DOCUMENTAÇÃO FRONTEND

8.1. EXPERIÊNCIA DO USUÁRIO

A experiência do usuário (UX) é um fator essencial no desenvolvimento de sistemas web, especialmente em aplicações voltadas ao público geral. Um sistema com boa usabilidade permite que o usuário execute suas tarefas de forma simples, intuitiva e eficiente, reduzindo erros e aumentando a satisfação durante a utilização da plataforma.

No contexto deste projeto, foram aplicados diversos conceitos de UX com o objetivo de tornar o processo de reserva mais organizado, acessível e visualmente intuitivo. A interface foi estruturada utilizando cards para exibição dos produtos, implementados pela classe `.card-container`, que utiliza o recurso `grid-template-columns` para organizar automaticamente os itens na tela. Essa abordagem facilita a visualização dos sabores disponíveis e melhora a navegação do usuário.

Também foram implementados controles visuais de quantidade por meio da classe `.qty-control`, utilizando botões de incremento e decremento (`.btn-mais` e `.btn-menos`) ao lado do campo de quantidade (`.input-qty`). Esse modelo de interação torna o processo mais rápido e intuitivo, reduzindo erros de digitação e melhorando a experiência do usuário durante a seleção dos produtos.

Outro recurso importante é o calendário com sistema de cores baseado em semáforo, implementado por meio das classes `.livre`, `.alerta` e `.esgotado`. Cada status possui cores específicas para indicar visualmente a disponibilidade dos carrinhos, permitindo que o usuário compreenda rapidamente quais datas estão disponíveis ou indisponíveis. Além disso, o sistema utiliza `pointer-events: none` nas datas esgotadas, impedindo interações inválidas e evitando erros durante o processo de reserva.

O sistema também utiliza feedbacks visuais importantes para melhorar a interação do usuário. A classe `.selecionado` destaca visualmente a data escolhida no calendário utilizando bordas e efeito de escala (`transform: scale(1.05)`), facilitando a identificação das opções selecionadas. Além disso, o modal de finalização da reserva (`.modal`) centraliza as informações e mantém o foco do usuário na conclusão do processo.

Outro aspecto relevante é a organização visual semelhante a um cupom fiscal, implementada pela classe `.nota-fiscal`, que utiliza fonte monoespaçada (Courier New) e

estrutura padronizada para facilitar a leitura e conferência das informações da reserva antes da finalização do pedido.

Também foi considerado o conceito de responsividade, permitindo que o sistema seja utilizado em diferentes dispositivos, como smartphones, tablets e computadores. O uso de grids flexíveis e medidas adaptáveis, como `minmax(220px, 1fr)` e `width: 90%`, permite que os elementos da interface se ajustem automaticamente a diferentes tamanhos de tela, tornando a aplicação mais acessível e funcional.

Além disso, foram utilizados elementos visuais modernos, como sombras suaves (`box-shadow`), bordas arredondadas (`border-radius`) e espaçamentos padronizados, proporcionando uma interface mais limpa, agradável e intuitiva para o usuário.

Dessa forma, as melhorias implementadas no sistema seguem princípios de design centrado no usuário, priorizando facilidade de aprendizado, eficiência de navegação, redução de erros, acessibilidade e satisfação durante a utilização da aplicação.

8.2. HTML

O HTML (HyperText Markup Language) é a linguagem responsável pela estruturação das páginas do sistema, organizando os elementos que serão exibidos ao usuário, como textos, imagens, formulários, menus e botões. No sistema de reservas de carrinhos de sorvete, o HTML foi utilizado para estruturar toda a interface da aplicação, permitindo a integração entre conteúdo, estilização e funcionalidades.

A estrutura da página foi organizada utilizando elementos semânticos que facilitam a compreensão e manutenção do código. O cabeçalho da aplicação contém a logomarca da empresa e o menu de navegação, permitindo que o usuário acesse rapidamente as principais seções do sistema.

A área principal do sistema foi estruturada para exibir os produtos disponíveis utilizando containers e cards individuais. Cada card apresenta informações importantes sobre os sabores de sorvete, incluindo imagem, nome do produto, preço e controles de quantidade. Essa organização melhora a visualização dos itens e facilita a interação do usuário durante o processo de seleção.

Também foram utilizados formulários HTML para captura das informações do cliente, como nome, telefone, endereço e detalhes do evento. Os campos foram organizados utilizando labels e inputs, garantindo melhor acessibilidade e compreensão durante o preenchimento das informações.

O sistema possui uma seção de calendário para seleção de datas disponíveis para reserva. Cada dia é representado por elementos organizados em grid, permitindo que o usuário visualize rapidamente as datas disponíveis, em alerta ou indisponíveis.

Outro recurso importante é a utilização de modais para exibição de informações importantes e finalização do pedido. Os modais centralizam o conteúdo na tela e permitem que o usuário revise os dados da reserva antes da confirmação final.

Além disso, o HTML também foi utilizado para estruturar o rodapé da aplicação, contendo informações de contato, links rápidos e dados institucionais da empresa. Essa organização melhora a navegação e fornece informações adicionais aos usuários.

A estrutura HTML foi desenvolvida de forma compatível com dispositivos móveis, permitindo integração com recursos de responsividade implementados no CSS. Dessa forma, a aplicação consegue manter organização e acessibilidade em diferentes tamanhos de tela.

Portanto, o HTML desempenha papel fundamental na construção da interface do sistema, organizando os elementos visuais e permitindo integração eficiente com CSS e JavaScript para proporcionar uma experiência completa ao usuário.

8.3. CSS

O CSS (Cascading Style Sheets) é responsável pela estilização visual do sistema de aluguel de carrinhos de sorvete, permitindo definir cores, layouts, responsividade, espaçamentos, animações e organização dos elementos exibidos na interface. No projeto, o CSS foi desenvolvido utilizando uma estrutura moderna baseada em variáveis globais, grids responsivos, flexbox e media queries, proporcionando uma experiência visual agradável, organizada e adaptável para diferentes dispositivos.

Inicialmente, foram definidas variáveis globais na pseudo-classe `:root`, permitindo centralizar toda a identidade visual do sistema. Foram criadas variáveis como `--oggi-roxo`, `--oggi-laranja`, `--oggi-rosa` e `--oggi-azul`, que representam as cores principais da marca Oggi Sorvetes. Além disso, também foram definidos padrões de cores secundárias, fundos, bordas e textos, facilitando futuras manutenções e garantindo padronização visual em toda a aplicação.

A estrutura global da página foi configurada através da estilização da tag `body`, onde foram definidos fonte padrão, cor de fundo, cores de texto, remoção de margens e paddings automáticos do navegador. A classe `.container` foi utilizada para centralizar o conteúdo principal da aplicação e limitar a largura máxima da página, garantindo melhor organização visual em telas grandes e pequenas.

O cabeçalho fixo do sistema foi implementado utilizando a classe `.header-fixa`, com

position: sticky, permitindo que o menu permaneça visível durante toda a navegação do usuário. Também foram utilizados efeitos modernos como backdrop-filter: blur(), box-shadow e bordas suaves, proporcionando aparência mais profissional e moderna. O menu de navegação utiliza display: flex para alinhamento horizontal dos links e possui adaptação automática para dispositivos móveis através do botão .menu-toggle e das media queries.

A seção principal do sistema conta com uma área hero estilizada pelas classes .hero-loja e .hero-loja-conteudo, utilizando CSS Grid para dividir o conteúdo em duas colunas. Essa seção foi projetada para destacar informações importantes da empresa, incluindo textos promocionais, botões de ação e imagem principal da loja. Também foram implementados efeitos visuais de sombras, arredondamentos e responsividade automática para dispositivos móveis.

As informações rápidas do sistema foram estilizadas pela classe .info-rapida, utilizando grid-template-columns para organizar os cards em colunas responsivas. Cada card utiliza efeitos visuais como bordas arredondadas, sombras suaves e barras coloridas superiores, criadas com pseudo-elementos ::before. Essa abordagem melhora a organização das informações e facilita a leitura dos conteúdos apresentados ao usuário.

Os cards de sabores foram implementados utilizando a classe .card-container, que utiliza display: grid com repeat(auto-fill, minmax()) para distribuir automaticamente os produtos conforme o tamanho da tela. Cada card individual possui estilizações modernas com efeitos hover, sombras, gradientes suaves e transições de animação. As imagens dos produtos utilizam object-fit: contain para manter a proporção correta das embalagens exibidas.

O controle de quantidade dos produtos foi desenvolvido através das classes .qty-control, .btn-mais, .btn-menos e .input-qty. Os botões utilizam cores distintas para facilitar a identificação visual das ações de aumentar ou diminuir quantidades. Também foram aplicados efeitos de hover, arredondamentos e alinhamentos centralizados para melhorar a usabilidade do componente.

O sistema de calendário de disponibilidade foi totalmente estilizado utilizando grids responsivos através da classe .calendar-grid. Cada dia do calendário utiliza a classe .dia-calendario, recebendo cores diferentes conforme sua disponibilidade. As classes .livre, .alerta e .esgotado representam visualmente os níveis de ocupação dos carrinhos. Datas indisponíveis utilizam pointer-events: none para impedir interações inválidas. Também foram implementados efeitos visuais para destacar a data selecionada pelo usuário através da classe .selecionado.

A seção de quantidade de carrinhos foi estilizada pela classe .secao-carrinhos, utilizando alinhamentos centralizados, bordas arredondadas e sombras suaves. O seletor de quantidade utiliza efeitos de foco personalizados através de box-shadow e mudanças de borda,

proporcionando melhor experiência visual durante a interação.

O botão principal de finalização da reserva utiliza a classe `.btn-finalizar`, apresentando destaque visual através de cores vibrantes, sombras e animações de hover. Esse botão possui estados diferentes para ações normais e desabilitadas, melhorando a comunicação visual com o usuário.

A seção de dúvidas frequentes foi implementada utilizando elementos `details` e `summary` estilizados pelas classes `.faq-lista` e `.faq-section`. Foram aplicados efeitos de expansão, mudanças de cor e ícones dinâmicos de abertura e fechamento utilizando pseudo-elementos `::after`. Essa abordagem permite criar uma área interativa e organizada para apresentação das perguntas frequentes do sistema.

Os modais de reserva e contrato foram estilizados pelas classes `.modal` e `.modal-content`, utilizando `position: fixed` para centralização na tela e `backdrop-filter: blur()` para escurecimento do fundo da página. Os formulários internos utilizam campos personalizados com bordas arredondadas, efeitos de foco e validações visuais, melhorando a experiência de preenchimento das informações.

O modal de contrato utiliza componentes específicos como `.contrato-scroll`, permitindo rolagem interna do conteúdo jurídico sem comprometer a navegação da página principal. Também foram estilizados os botões de aceite do contrato e envio para WhatsApp, reforçando visualmente as ações mais importantes do processo de reserva.

O rodapé da aplicação foi estruturado utilizando CSS Grid através da classe `.footer-conteudo`, organizando as informações institucionais em colunas responsivas. Foram utilizados gradientes, destaques visuais e links interativos para melhorar a apresentação das informações de contato, endereço e horário de funcionamento da empresa.

Além disso, o sistema utiliza diversas media queries para adaptação automática da interface em diferentes tamanhos de tela. Recursos como grids flexíveis, alterações de layout, redimensionamento de fontes e reorganização de elementos garantem que o sistema funcione corretamente em computadores, tablets e smartphones.

Dessa forma, o CSS desempenha papel fundamental no sistema, sendo responsável pela criação de uma interface moderna, organizada, intuitiva, responsiva e visualmente alinhada com a identidade da marca Oggi Sorvetes, contribuindo diretamente para a usabilidade e experiência dos usuários.

8.4. JAVASCRIPT

O JavaScript é responsável por toda a lógica dinâmica e interativa do sistema de aluguel

de carrinhos de sorvete, permitindo o controle de reservas, gerenciamento de sabores, funcionamento do calendário, validações de formulários, integração com APIs do Django e comunicação com o WhatsApp. No projeto, o JavaScript foi desenvolvido utilizando recursos modernos da linguagem, como funções assíncronas, manipulação de DOM, eventos, arrays, objetos e requisições HTTP utilizando Fetch API, proporcionando uma aplicação dinâmica, responsiva e integrada ao backend.

Inicialmente, foram definidas variáveis globais responsáveis por armazenar os dados principais do sistema durante a execução da aplicação. Variáveis como `dataSelecionada`, `totalCarrinhosGlobal`, `ocupacaoGlobal`, `saboresCache` e `carrinho` são utilizadas para controlar o estado atual da reserva, armazenar os sabores carregados da API, controlar a ocupação do calendário e registrar os produtos selecionados pelo usuário. Também foram criadas variáveis responsáveis pelo controle de navegação mensal do calendário, permitindo visualizar diferentes períodos de disponibilidade.

A inicialização do sistema ocorre através do evento `DOMContentLoaded`, garantindo que todo o conteúdo HTML esteja carregado antes da execução das funções principais. Nesse momento são chamadas as funções `carregarSabores()` e `carregarMes()`, responsáveis por buscar os sabores cadastrados no backend e renderizar o calendário inicial do sistema.

O carregamento dos sabores foi implementado através da função assíncrona `carregarSabores()`, que realiza uma requisição HTTP utilizando `fetch()` para o endpoint da API Django responsável pelos sabores. Após receber os dados em formato JSON, os sabores são armazenados em cache na variável `saboresCache` e enviados para a função `renderizarCards()`, responsável pela construção dinâmica dos cards de produtos exibidos na interface.

A função `renderizarCards()` realiza a criação dinâmica dos elementos HTML dos sabores utilizando template literals. Cada card contém imagem do produto, nome, preço e controles de quantidade. Os controles utilizam botões de incremento e decremento, além de campos de entrada numérica, permitindo ao usuário alterar manualmente a quantidade desejada de cada sabor.

O controle de quantidades foi implementado através das funções `atualizarQtd()` e `inputManual()`. Essas funções realizam validações de integridade, impedindo valores negativos ou superiores ao limite estabelecido. Após cada alteração, o sistema atualiza automaticamente o resumo da reserva utilizando a função `atualizarResumoReserva()`, garantindo sincronização entre os dados selecionados e as informações exibidas na tela.

A função `atualizarResumoReserva()` é responsável por gerar dinamicamente o resumo financeiro do pedido. O sistema percorre os sabores selecionados, calcula subtotais individuais,

soma os valores dos produtos e aplica automaticamente a lógica de aluguel dos carrinhos. Caso o subtotal dos sorvetes ultrapasse determinado valor, o aluguel pode ser gratuito, seguindo a regra de negócio implementada também no backend em Python. O resumo final exibe produtos selecionados, data escolhida, quantidade de carrinhos e valor total da reserva.

O processo de finalização do pedido foi desenvolvido através da função `finalizarPedido()`. Essa função realiza validações obrigatórias, verificando se o usuário selecionou uma data e pelo menos um sabor antes de continuar. Em seguida, os dados preenchidos no formulário são capturados e inseridos dinamicamente no modal de contrato, permitindo que o cliente visualize todas as informações da reserva antes do envio final.

A integração com o backend Django ocorre através da função assíncrona `enviarFormularioServidor()`. Nessa função é criado um objeto payload contendo todos os dados da reserva, incluindo cliente, telefone, endereço, sabores selecionados, quantidade de carrinhos e data escolhida. Esses dados são enviados via método POST utilizando Fetch API para o endpoint da API responsável pela criação das reservas.

Durante o envio da requisição, também é utilizado o token CSRF do Django, obtido através da função `getCookie()`, garantindo maior segurança contra ataques CSRF (Cross-Site Request Forgery). Após o processamento da reserva no backend, o sistema redireciona automaticamente o usuário para o WhatsApp utilizando a URL retornada pela API, facilitando o contato direto com a empresa.

O sistema também possui funções auxiliares de utilidade, como `maskarTelefone()`, responsável por aplicar formatação automática ao número de telefone digitado pelo usuário. Essa função remove caracteres inválidos, limita a quantidade de dígitos e aplica a máscara padrão brasileira no formato de WhatsApp.

O calendário de disponibilidade foi desenvolvido utilizando integração dinâmica com a API do Django através da função `carregarDisponibilidade()`. Essa função consulta o backend para obter a quantidade total de carrinhos disponíveis e o nível de ocupação de cada data. Caso ocorra falha de comunicação com o servidor, o sistema possui tratamento de erros utilizando `try/catch`, garantindo funcionamento mínimo da aplicação.

A classificação visual da ocupação é realizada pela função `definirClasseOcupacao()`, que calcula o percentual de ocupação dos carrinhos disponíveis para cada data. Dependendo do percentual calculado, o sistema atribui classes como livre, alerta ou esgotado, permitindo alteração visual automática das cores do calendário.

A renderização dinâmica do calendário ocorre através da função `renderizarCalendario()`, responsável pela criação completa da grade mensal. O sistema calcula automaticamente o

primeiro dia da semana do mês, gera os espaços vazios necessários para alinhamento correto e cria cada dia do calendário dinamicamente. Datas passadas são automaticamente bloqueadas, impedindo reservas retroativas.

Quando o usuário seleciona uma data disponível, o sistema aplica destaque visual na data escolhida, armazena a dataSelecionada e atualiza automaticamente a disponibilidade de carrinhos através da função `atualizarSeletorCarrinhos()`. Essa função calcula quantos carrinhos ainda estão disponíveis naquela data específica e preenche dinamicamente o seletor de quantidade de carrinhos.

A navegação entre meses foi implementada através das funções `carregarMes()`, `mudarMes()` e `atualizarLabelMesAno()`. Essas funções permitem avançar ou retroceder entre os meses do calendário, atualizando automaticamente os dados de disponibilidade vindos da API. O sistema também impede navegação para meses anteriores ao atual, aumentando a integridade das reservas.

Os modais do sistema são controlados pelas funções `abrirModalContrato()` e `fecharModalContrato()`, responsáveis por exibir ou ocultar a janela de contrato. O aceite do contrato é controlado pela função `alternarBotaoFinalizar()`, que habilita o botão de envio apenas após o usuário marcar a confirmação de aceite dos termos.

O sistema também implementa funcionalidades de menu responsivo para dispositivos móveis. Através do uso de `addEventListener()`, o botão `.menu-toggle` permite abrir e fechar o menu de navegação em telas menores. Além disso, ao clicar em qualquer link do menu, o sistema fecha automaticamente o menu mobile, melhorando a experiência de navegação.

Dessa forma, o JavaScript desempenha papel fundamental no funcionamento do sistema, sendo responsável pela lógica de negócio, interatividade da interface, comunicação com o backend Django, gerenciamento de reservas, controle de disponibilidade e integração com o WhatsApp, proporcionando uma aplicação moderna, dinâmica, segura e totalmente funcional para o gerenciamento de aluguel de carrinhos de sorvete da Oggi Sorvetes.

9. DOCUMENTAÇÃO BACKEND

9.1. API

A API do sistema foi desenvolvida utilizando o framework Django e tem como principal objetivo realizar a comunicação entre o frontend e o backend da aplicação. Ela é responsável por fornecer informações ao sistema, processar dados enviados pelos usuários e executar operações relacionadas às reservas de carrinhos de sorvete. A estrutura da API foi organizada

em diferentes funções responsáveis pelo gerenciamento de sabores, controle de disponibilidade e criação de reservas.

A função `api_sabores()` foi desenvolvida para retornar todos os sabores ativos cadastrados no sistema. Inicialmente, a API realiza uma consulta no banco de dados utilizando o model `Sorvete`, filtrando apenas os registros que possuem o atributo `ativo=True`. Após a consulta, os dados são organizados em formato JSON contendo o identificador do sabor, nome, preço e URL da imagem correspondente. O preço é convertido para string antes de ser enviado ao frontend, garantindo maior precisão no tratamento de valores monetários pelo JavaScript. Essa API permite que os sabores sejam carregados dinamicamente na interface do sistema, melhorando a experiência do usuário e reduzindo a necessidade de atualização manual da página.

A função `api_disponibilidade()` é responsável pelo controle de disponibilidade dos carrinhos de sorvete. Essa API recebe como parâmetro o mês selecionado pelo usuário e realiza uma consulta no banco de dados para verificar a quantidade de reservas existentes em cada data. Inicialmente, o sistema separa o ano e o mês informados na requisição. Em seguida, realiza a contagem dos carrinhos ativos cadastrados utilizando o model `Carrinho`. Posteriormente, a API consulta todas as reservas do mês selecionado, desconsiderando aquelas que possuem status “cancelado”. O sistema utiliza o recurso `annotate(Count('id'))` para contabilizar automaticamente a quantidade de reservas por data. Após o processamento das informações, a API retorna um objeto JSON contendo o total de carrinhos disponíveis e a quantidade de reservas registradas em cada dia do mês. Esses dados são utilizados pelo frontend para alimentar o calendário visual da aplicação, permitindo aplicar o sistema de semáforo que identifica datas livres, em alerta ou esgotadas. Além disso, a função possui tratamento de erros utilizando `try/except`, garantindo que datas inválidas retornem mensagens apropriadas ao usuário.

Já a função `api_criar_reserva()` é responsável pelo processamento completo das reservas realizadas pelos clientes. Essa API recebe requisições do tipo POST contendo os dados enviados pelo frontend em formato JSON. Inicialmente, os dados da requisição são convertidos para objetos Python utilizando `json.loads(request.body)`. Em seguida, o sistema realiza o cadastro do cliente através do model `Cliente`, armazenando informações como nome, telefone, endereço e e-mail. O campo de e-mail possui um tratamento específico para impedir que valores vazios sejam armazenados incorretamente no banco de dados, convertendo strings vazias para NULL.

Após a criação do cliente, o sistema realiza a criação inicial da reserva utilizando o model `Reserva`. Nesse momento, a reserva é criada sem os itens vinculados, permitindo que o

banco de dados gere automaticamente a chave primária necessária para os relacionamentos posteriores. Em seguida, a API percorre todos os sabores selecionados pelo usuário utilizando uma estrutura de repetição `for`. Para cada item selecionado, o sistema busca o sabor correspondente no banco de dados utilizando o `model Sorvete`. Após localizar o produto, é criado um relacionamento na tabela `ReservaProduto`, responsável por armazenar os itens associados à reserva juntamente com suas quantidades.

Depois que todos os itens são vinculados à reserva, o sistema atualiza o valor total do pedido utilizando os dados recebidos pelo frontend. Ao finalizar o processamento, a API retorna uma resposta JSON contendo o status da operação e um link automático para contato via WhatsApp. Esse link é gerado através do método `gerar_link_whatsapp()`, permitindo facilitar a comunicação entre o cliente e a empresa após a realização da reserva.

Além das funcionalidades principais, a API também possui mecanismos de tratamento de erros para garantir maior estabilidade e segurança durante a execução do sistema. Caso algum sabor selecionado não seja encontrado no banco de dados, a exceção `Sorvete.DoesNotExist` é acionada e uma mensagem de erro apropriada é retornada ao usuário. Também foi implementado um tratamento genérico utilizando `except Exception as e`, permitindo capturar falhas inesperadas sem interromper completamente o funcionamento da aplicação.

A comunicação entre frontend e backend ocorre de forma dinâmica através das APIs desenvolvidas. O frontend realiza requisições assíncronas para buscar sabores, verificar disponibilidade e registrar reservas sem necessidade de recarregamento da página. Essa integração contribui diretamente para a melhoria da usabilidade, proporcionando respostas mais rápidas, maior interatividade e melhor experiência para os usuários do sistema.

9.2. MODELS

A camada de Models do sistema foi desenvolvida utilizando o framework Django, sendo responsável pela estruturação, armazenamento, validação e gerenciamento das informações do sistema de aluguel de carrinhos de sorvete. Os models representam as tabelas do banco de dados e concentram as principais regras de negócio da aplicação, incluindo controle de estoque, cálculo financeiro, validação de reservas, disponibilidade de carrinhos e integração com WhatsApp.

O `model Carrinho` foi criado para representar os carrinhos disponíveis para locação no sistema. Ele possui o campo `preco_diaria`, responsável por armazenar o valor da diária do carrinho utilizando o tipo `DecimalField`, garantindo maior precisão em operações financeiras.

Também foi implementado o campo `status`, utilizado para controlar se o carrinho está ativo ou indisponível para locação. O método `_str_()` retorna o identificador do carrinho, facilitando sua visualização dentro do painel administrativo do Django.

O model `Sorvete` é responsável pelo cadastro dos sabores disponíveis no sistema. Foram implementados os campos `nome_sorvete`, `preco`, `quantidade`, `imagem` e `ativo`. O campo `nome_sorvete` armazena o nome do produto, enquanto `preco` registra seu valor unitário. O campo `quantidade` controla o estoque disponível de cada sabor. Já o campo `imagem` utiliza `ImageField`, permitindo upload de imagens dos produtos diretamente no sistema.

Dentro do model `Sorvete` também foi implementado o método `save()`, responsável por executar processos automáticos sempre que um produto é salvo. Nesse método, a biblioteca `Pillow` é utilizada para abrir e redimensionar automaticamente as imagens cadastradas, limitando o tamanho máximo para 400x400 pixels. Essa funcionalidade melhora o desempenho do sistema e reduz o consumo de armazenamento no servidor.

O model `Cliente` foi desenvolvido para armazenar as informações dos usuários responsáveis pelas reservas. Ele possui os campos `nome_cliente`, `endereco`, `telefone` e `email`. Essas informações são utilizadas tanto para exibição administrativa quanto para comunicação direta com os clientes durante o processo de confirmação das reservas.

O model `Reserva` representa a principal entidade do sistema, sendo responsável pelo gerenciamento completo dos pedidos de aluguel. Foram implementados diversos campos importantes, como `id_cliente`, que cria relacionamento com o model `Cliente` através de `ForeignKey`, e `id_carrinho`, que relaciona a reserva ao carrinho alugado. O campo `data_evento` armazena a data da reserva, enquanto `valor_pedido` registra o valor total calculado automaticamente pelo sistema.

Também foi implementado o campo `status`, que utiliza `STATUS_CHOICES` para controlar os estados possíveis da reserva, sendo eles “pendente”, “confirmado” e “cancelado”. O campo `descricao` permite armazenar observações adicionais do cliente, enquanto `disponibilidade` auxilia no controle interno de disponibilidade dos carrinhos.

O método `clean()` foi desenvolvido para aplicar regras de validação antes do salvamento da reserva no banco de dados. A principal regra implementada é o prazo mínimo de 24 horas para realização das reservas. O sistema calcula automaticamente a data mínima permitida utilizando `timedelta(days=1)`. Caso o cliente tente reservar uma data inválida, o sistema lança uma exceção utilizando `ValidationError`, impedindo o cadastro da reserva.

Além disso, o método `clean()` também possui uma verificação de bloqueio para reservas já confirmadas, permitindo maior controle administrativo sobre alterações em pedidos

concluídos.

O método `save()` do model `Reserva` foi personalizado para recalcular automaticamente o valor total do pedido sempre que a reserva for atualizada. Caso a reserva ainda esteja pendente ou sem valor definido, o sistema executa o método `total_pedido()`, garantindo consistência financeira automática.

O método `baixar_estoque_real()` é responsável por realizar a baixa automática do estoque dos sorvetes após confirmação da reserva. O sistema percorre todos os itens vinculados ao pedido através do relacionamento `itens.all()`, reduzindo a quantidade disponível de cada sabor conforme a quantidade escolhida pelo cliente.

Para os cálculos financeiros, foram implementados diversos métodos auxiliares. O método `subtotal_sorvetes()` calcula apenas o valor total dos produtos selecionados pelo cliente. Já o método `taxa_aluguel()` calcula automaticamente a taxa de locação dos carrinhos. Caso o subtotal dos produtos seja igual ou superior a R\$300,00, o aluguel é concedido gratuitamente. Caso contrário, o sistema busca automaticamente o valor da diária do carrinho ativo cadastrado no banco de dados.

O método `total_pedido()` realiza a soma do subtotal dos sorvetes com a taxa de aluguel, retornando o valor final da reserva. Dessa forma, toda a lógica financeira permanece centralizada diretamente no model, garantindo maior organização e segurança da aplicação.

O sistema também possui integração automática com WhatsApp através do método `gerar_link_whatsapp()`. Esse método gera dinamicamente uma mensagem contendo número da reserva, data do evento, valores financeiros e informações adicionais sobre frete. A mensagem é codificada utilizando `urllib.parse.quote()`, permitindo gerar automaticamente um link funcional para envio direto via WhatsApp.

O método de classe `vagas_disponiveis()` foi criado para controlar a quantidade de carrinhos livres em uma determinada data. O sistema calcula quantos carrinhos estão ativos e quantas reservas confirmadas já existem para o mesmo dia, retornando automaticamente a quantidade restante disponível para novas reservas.

O método `pode_confirmar()` utiliza a função de disponibilidade para verificar se ainda existem vagas disponíveis antes da confirmação definitiva da reserva. Isso impede overbooking e garante controle adequado dos carrinhos disponíveis.

Também foi implementado o método `verificar_alerta_estoque()`, responsável por identificar possíveis insuficiências de estoque nos produtos selecionados. O sistema percorre todos os itens da reserva e compara as quantidades solicitadas com o estoque disponível, gerando alertas administrativos caso exista falta de produtos.

O model `ReservaProduto` foi criado para representar os itens individuais vinculados às reservas. Esse model utiliza relacionamento `ForeignKey` tanto para `Reserva` quanto para `Sorvete`, permitindo associar múltiplos sabores a uma única reserva. O campo `quantidade_escolhida` armazena quantas unidades de cada sabor foram selecionadas pelo cliente.

A utilização do relacionamento `related_name="itens"` facilita o acesso aos produtos vinculados diretamente através da reserva, permitindo que métodos financeiros, estoque e validações percorram facilmente todos os itens relacionados ao pedido.

Dessa forma, a camada de Models desempenha papel fundamental no sistema, sendo responsável pela estruturação completa do banco de dados, implementação das regras de negócio, cálculos automáticos, controle de estoque, validações de segurança, gerenciamento financeiro e integração operacional do sistema de aluguel de carrinhos da Oggi Sorvetes.

9.3. VIEWS

A camada de Views do sistema foi desenvolvida utilizando o framework Django e possui como principal responsabilidade controlar a comunicação entre o front-end, os models e o banco de dados. As views funcionam como intermediadoras das requisições realizadas pelos usuários, processando informações, executando regras de negócio e retornando respostas adequadas para a interface do sistema de aluguel de carrinhos de sorvete.

Inicialmente, foi utilizada a função `render` da biblioteca `django.shortcuts`, responsável por carregar e exibir páginas HTML ao usuário. Também foi importada a classe `JsonResponse`, utilizada para retornar dados em formato JSON para integração com o JavaScript do sistema. Além disso, foram importados os models `Carrinho` e `Reserva`, permitindo que as views acessem diretamente os dados armazenados no banco de dados.

A view `index()` foi criada como a página principal da aplicação. Sua principal função é renderizar o template `index.html`, localizado dentro da pasta `templates/aluguel/`. Essa página concentra toda a interface do sistema, incluindo calendário de disponibilidade, listagem de sabores, formulários de reserva, contrato digital e integração com o WhatsApp.

A view `index()` recebe o objeto `request`, que representa a requisição HTTP enviada pelo navegador do usuário. Em seguida, a função `render()` é utilizada para retornar a página HTML completamente processada pelo servidor. Dessa forma, o Django realiza a integração entre o back-end e a interface visual exibida no navegador.

A principal funcionalidade da camada de views está implementada na função `api_disponibilidade()`, responsável por fornecer ao front-end todas as informações de

disponibilidade dos carrinhos em tempo real. Essa view funciona como uma API interna do sistema, permitindo comunicação assíncrona entre o JavaScript e o servidor utilizando requisições AJAX via `fetch()`.

Inicialmente, a função captura o parâmetro `mes` enviado pela URL através do comando `request.GET.get('mes', None)`. Esse parâmetro representa o mês e ano selecionados pelo usuário no calendário do sistema, sendo utilizado para filtrar as reservas correspondentes no banco de dados.

Após receber o mês solicitado, o sistema executa a contagem total de carrinhos cadastrados utilizando `Carrinho.objects.count()`. Essa consulta retorna a quantidade total de carrinhos disponíveis no sistema, servindo como base para o cálculo de ocupação e disponibilidade das datas.

Em seguida, a view realiza uma busca no banco de dados utilizando o model `Reserva`. A consulta `Reserva.objects.filter(data_evento__startswith=mes_ano)` retorna todas as reservas relacionadas ao mês selecionado pelo usuário. O operador `startswith` permite filtrar registros cuja data começa com o padrão informado, facilitando a separação das reservas de cada mês.

Após recuperar as reservas do banco de dados, o sistema cria um dicionário chamado `ocupacao`, responsável por armazenar matematicamente quantos carrinhos estão reservados em cada dia específico do mês.

A view percorre todas as reservas utilizando um laço `for reserva in reservas_do_mes`. Durante cada repetição, a data do evento é convertida para o padrão `YYYY-MM-DD` através do método `strftime('%Y-%m-%d')`. Essa formatação é necessária porque o calendário em JavaScript utiliza esse mesmo padrão para identificar as datas corretamente.

Dentro do loop, o sistema utiliza o método `ocupacao.get(data_str, 0)` para recuperar quantos carrinhos já estão ocupados naquela data específica. Em seguida, é somada a quantidade de carrinhos alugados da reserva atual através do atributo `reserva.quantidade_carrinhos`.

Essa lógica permite calcular automaticamente o nível de ocupação diária dos carrinhos, fornecendo ao calendário do sistema informações atualizadas em tempo real sobre disponibilidade, lotação parcial e datas esgotadas.

Após concluir os cálculos, a view monta um dicionário chamado `dados`, contendo duas informações principais: `total_carrinhos`, que representa a quantidade total de carrinhos cadastrados no sistema, e `ocupacao`, que armazena o mapa completo de ocupação das datas.

Por fim, os dados são retornados ao front-end utilizando `JsonResponse(dados)`. Essa resposta em formato JSON é consumida diretamente pelo JavaScript da aplicação, permitindo

atualizar dinamicamente o calendário visual sem necessidade de recarregar a página inteira.

A integração entre essa view e o front-end permite que o sistema altere automaticamente as cores dos dias no calendário, exibindo visualmente se uma data está livre, parcialmente ocupada ou totalmente indisponível. Dessa forma, o usuário consegue visualizar a disponibilidade dos carrinhos em tempo real durante o processo de reserva.

Além disso, a utilização de APIs internas em formato JSON melhora significativamente a performance da aplicação, reduzindo recarregamentos completos da página e proporcionando uma experiência mais rápida, moderna e interativa para os usuários.

Dessa forma, a camada de Views desempenha papel fundamental no sistema, sendo responsável pela renderização da interface principal, comunicação entre cliente e servidor, processamento das consultas de disponibilidade, integração com o banco de dados e fornecimento dinâmico das informações utilizadas pelo calendário inteligente do sistema de aluguel da Oggi Sorvetes.

9.4. TEMPLATE

A camada de Templates do sistema foi desenvolvida utilizando o mecanismo de templates do framework Django e possui como principal responsabilidade apresentar a interface visual do sistema aos usuários. Os templates atuam como a camada de apresentação da arquitetura MVT (Model-View-Template), sendo responsáveis por exibir informações processadas pelas views e permitir a interação dos usuários com as funcionalidades do sistema de aluguel de carrinhos de sorvete.

Inicialmente, o template utiliza a instrução `{% load static %}`, responsável por habilitar o carregamento de arquivos estáticos do Django. Esse recurso permite acessar folhas de estilo CSS, arquivos JavaScript e imagens armazenadas na pasta static da aplicação, garantindo a organização dos recursos visuais e funcionais do sistema.

A estrutura principal do template foi construída utilizando HTML5, permitindo a criação de uma interface moderna, responsiva e compatível com diferentes dispositivos. O arquivo representa a página principal da aplicação e concentra todas as etapas do processo de reserva dos carrinhos de sorvete.

O cabeçalho da página foi desenvolvido através da estrutura `<header>`, contendo a logomarca da Oggi Sorvetes e um menu de navegação responsivo. Esse menu permite que o usuário acesse rapidamente as principais seções do sistema, incluindo informações sobre o funcionamento do aluguel, seleção de sabores, calendário de disponibilidade, dúvidas frequentes e canais de contato.

Logo após o cabeçalho, foi implementada uma seção de apresentação denominada Hero Section. Essa área possui a função de destacar o serviço oferecido pela empresa, apresentando uma descrição resumida do aluguel de carrinhos de sorvete, além de botões de ação que direcionam o usuário para as etapas principais da reserva. Também é exibida uma imagem da fachada da loja, fortalecendo a identidade visual da empresa.

Na sequência, o template apresenta uma seção informativa contendo as principais características do serviço. Nessa área são exibidas informações sobre retirada e devolução dos carrinhos, capacidade média de armazenamento de sorvetes, tempo estimado de refrigeração e opções de entrega ou retirada na loja. Essas informações auxiliam o cliente na compreensão das condições de utilização do equipamento.

A primeira etapa do processo de reserva é representada pela seção de seleção de sabores. Nessa área existe um elemento HTML identificado como lista-sorvetes, que inicialmente permanece vazio. Seu conteúdo é preenchido dinamicamente através do arquivo JavaScript principal da aplicação, responsável por carregar e exibir os sabores disponíveis para locação.

A segunda etapa corresponde à seleção da data do evento. Para essa funcionalidade foi criada uma seção de calendário que permite ao usuário visualizar a disponibilidade dos carrinhos em cada dia do mês. O template contém botões para navegação entre meses, legenda de disponibilidade e uma grade de calendário que é preenchida dinamicamente pelo JavaScript.

As informações exibidas no calendário são obtidas através da comunicação com a API interna do sistema. Com base nos dados retornados pela camada de Views, o JavaScript altera automaticamente as cores dos dias exibidos, indicando visualmente se existem vagas disponíveis, poucas vagas restantes ou se a data já se encontra totalmente ocupada.

Após a escolha da data, o sistema exibe a seção de quantidade de carrinhos. Essa área permanece oculta inicialmente e é apresentada apenas quando uma data válida é selecionada. O usuário pode escolher a quantidade de carrinhos necessária através de uma lista suspensa, cuja capacidade máxima é definida conforme a disponibilidade retornada pelo servidor.

Na etapa seguinte foi implementada uma área destinada à revisão e finalização da reserva. Essa funcionalidade é acionada através do botão "Revisar e finalizar reserva", que abre uma janela modal contendo o resumo completo do pedido realizado pelo cliente.

O modal de reserva possui um formulário destinado à coleta das informações do solicitante. Entre os dados solicitados estão nome completo, telefone de contato via WhatsApp, endereço do evento, e-mail opcional e observações adicionais. Para garantir maior integridade das informações, diversos campos utilizam atributos de validação HTML, como `required` e `maxlength`, restringindo entradas inválidas ou excessivamente longas.

Após o preenchimento do formulário, o sistema apresenta um segundo modal contendo o contrato digital de comodato dos carrinhos de sorvete. Esse documento descreve as responsabilidades da empresa e do cliente durante o período de utilização do equipamento.

O contrato apresenta automaticamente informações preenchidas pelo usuário, incluindo endereço do evento, quantidade de carrinhos solicitados e data da reserva. Dessa forma, o documento é personalizado para cada solicitação realizada no sistema.

Para prosseguir com o processo de reserva, o usuário deve marcar uma caixa de seleção indicando que leu e concorda com todos os termos contratuais. Somente após essa confirmação o botão de envio é habilitado, garantindo o aceite formal das condições estabelecidas pela empresa.

O template também possui uma seção de Perguntas Frequentes (FAQ), construída utilizando a tag HTML `<details>`. Essa funcionalidade permite expandir e recolher respostas de forma dinâmica, melhorando a experiência de navegação e reduzindo dúvidas comuns relacionadas ao aluguel dos carrinhos.

Ao final da página foi implementado um rodapé institucional contendo informações de contato da empresa, endereço físico da loja, horários de funcionamento e links para os canais digitais da Oggi Sorvetes. Essa seção facilita a comunicação entre clientes e estabelecimento, além de fornecer informações complementares importantes.

Além da estrutura HTML, o template realiza integração direta com arquivos estáticos da aplicação. O arquivo `style.css` é responsável pela estilização visual da interface, enquanto o arquivo `main.js` implementa todas as funcionalidades dinâmicas do sistema, incluindo carregamento dos sabores, controle do calendário, exibição dos modais, validação de dados e integração com o WhatsApp.

Dessa forma, a camada de Templates desempenha papel fundamental no sistema, sendo responsável pela apresentação visual da aplicação, organização do fluxo de reserva, coleta das informações dos usuários, exibição das disponibilidades dos carrinhos e interação entre o cliente e as funcionalidades disponibilizadas pelo back-end desenvolvido em Django.

9.5. MIGRATION

A estrutura das migrations foi desenvolvida utilizando o mecanismo de versionamento de banco de dados fornecido pelo framework Django, permitindo o controle das alterações realizadas nos modelos da aplicação ao longo do desenvolvimento do sistema de aluguel de carrinhos de sorvete. As migrations funcionam como registros históricos das modificações efetuadas na estrutura do banco de dados, garantindo que todas as tabelas, relacionamentos e

campos sejam criados e atualizados de forma organizada e segura.

A migration inicial foi responsável pela criação das principais entidades do sistema, estabelecendo a base para o funcionamento da aplicação. Nessa etapa foram criadas as tabelas responsáveis pelo armazenamento dos carrinhos de sorvete, sabores disponíveis, clientes e reservas. Cada tabela recebeu seus respectivos campos, tipos de dados e restrições de integridade, permitindo que o banco de dados armazenasse corretamente as informações necessárias para o gerenciamento das locações.

A tabela de carrinhos foi estruturada para armazenar informações relacionadas aos equipamentos disponíveis para aluguel, incluindo o valor da diária e o status de disponibilidade. Essa estrutura permite controlar quais carrinhos podem ser disponibilizados para novas reservas e quais estão temporariamente indisponíveis para utilização.

A tabela de sorvetes foi criada com o objetivo de armazenar os sabores oferecidos pela empresa. Nela são registrados dados como nome do produto, preço unitário, quantidade em estoque, imagem ilustrativa e status de ativação. Essa estrutura possibilita que os sabores sejam exibidos dinamicamente na interface do sistema e permite o controle de estoque dos produtos disponíveis para locação.

A tabela de clientes foi desenvolvida para armazenar os dados cadastrais dos usuários que realizam reservas. Foram incluídos campos para nome, endereço, telefone e e-mail, permitindo a identificação do cliente e facilitando o contato para confirmação dos pedidos e organização logística dos eventos.

A tabela de reservas foi implementada como a principal entidade do sistema, sendo responsável por registrar todas as solicitações de aluguel realizadas pelos clientes. Essa estrutura armazena informações como cliente responsável, carrinho vinculado, data do evento, valor total do pedido, descrição adicional, status da reserva e disponibilidade. Além disso, foram definidos relacionamentos utilizando chaves estrangeiras (Foreign Keys), garantindo a integridade dos dados entre clientes, carrinhos e reservas.

Posteriormente, novas migrations foram criadas para aprimorar o modelo de dados e atender às necessidades identificadas durante o desenvolvimento do projeto. Entre essas alterações destaca-se a criação da tabela ReservaProduto, responsável por estabelecer o relacionamento entre reservas e sabores de sorvete. Essa implementação permitiu que uma única reserva pudesse conter diversos sabores diferentes, aumentando a flexibilidade do sistema e adequando-o ao cenário real de utilização.

Também foi adicionado o campo quantidade_escolhida na tabela ReservaProduto, permitindo registrar a quantidade específica de cada sabor selecionado pelo cliente. Essa

modificação foi fundamental para o cálculo correto do valor dos pedidos, controle de estoque e geração dos resumos apresentados durante a finalização da reserva.

As migrations também foram utilizadas para ajustes de regras de negócio, alterações de campos, inclusão de valores padrão e refinamento dos relacionamentos entre as entidades do sistema. Todas essas modificações foram registradas de forma sequencial, permitindo que o banco de dados evoluísse juntamente com a aplicação sem comprometer a consistência das informações armazenadas.

O processo de geração e aplicação das migrations foi realizado utilizando os comandos `makemigrations` e `migrate` do Django. O comando `makemigrations` foi responsável por identificar alterações realizadas nos modelos e gerar automaticamente os arquivos de migration correspondentes. Já o comando `migrate` executou essas alterações no banco de dados, criando ou atualizando as tabelas conforme necessário.

Além de facilitar a manutenção do sistema, o uso de migrations contribui para a segurança e confiabilidade da aplicação, uma vez que todas as alterações estruturais ficam registradas e podem ser reproduzidas em diferentes ambientes de desenvolvimento, testes e produção. Essa abordagem também favorece o trabalho colaborativo entre desenvolvedores, permitindo que todos utilizem a mesma estrutura de banco de dados de forma padronizada.

Dessa forma, a implementação das migrations desempenha um papel fundamental no sistema de aluguel de carrinhos de sorvete, garantindo o controle da evolução do banco de dados, a integridade das informações armazenadas e a correta integração entre os modelos responsáveis pelo gerenciamento de clientes, produtos, carrinhos e reservas.

10. DOCUMENTAÇÃO DO QUE FOI FEITO EM CADA SPRINT

10.1 SPRINT 1

Durante a Sprint 1, o foco foi o planejamento e estruturação inicial do sistema web de locação de carrinhos de sorvete da empresa Oggi Sorvetes, localizada no centro de Barueri. Nesta etapa, foram definidos os principais objetivos do projeto, bem como o escopo da aplicação, buscando compreender as necessidades da empresa e os problemas existentes no processo manual de reservas e controle de pedidos.

Inicialmente, foi desenvolvida a introdução, justificativa e definição dos objetivos gerais e específicos do trabalho. A justificativa teve como foco demonstrar a importância da Tecnologia da Informação na melhoria dos processos internos da empresa, destacando benefícios como automação, organização das informações, redução de erros e melhoria no atendimento ao cliente. Além disso, foram definidos os limites e funcionalidades que fariam

parte do escopo do sistema.

Também foram realizados os levantamentos e definição dos requisitos funcionais e não funcionais do sistema. Entre os principais requisitos definidos, destacam-se o cadastro de clientes, controle de reservas, gerenciamento de carrinhos e produtos, verificação de disponibilidade e aplicação das regras de negócio. Essa etapa foi fundamental para orientar o desenvolvimento das próximas funcionalidades do sistema.

Na área de modelagem, foram desenvolvidos os diagramas e modelos do banco de dados, incluindo os modelos conceitual, lógico e físico. O modelo conceitual permitiu representar as entidades e seus relacionamentos de forma abstrata, enquanto o modelo lógico estruturou as tabelas, atributos e relacionamentos do sistema. Já o modelo físico foi responsável pela implementação da estrutura do banco de dados utilizando SQL, garantindo a organização e integridade das informações armazenadas.

Além disso, foi criado o repositório do projeto no GitHub, permitindo o armazenamento, versionamento e compartilhamento do código entre os integrantes da equipe. A utilização do GitHub contribuiu para a organização do desenvolvimento, controle das alterações realizadas e centralização da documentação do projeto. Dessa forma, a Sprint 1 teve como principal objetivo estabelecer toda a base estrutural, documental e técnica necessária para o desenvolvimento das próximas etapas do sistema.

10.2 SPRINT 2

Durante a Sprint 2, a equipe concentrou esforços no desenvolvimento técnico e estrutural do sistema, iniciando a implementação prática do frontend e backend da aplicação. Nesta etapa, foram definidos os principais padrões arquiteturais e tecnologias utilizadas no projeto, buscando garantir maior organização, escalabilidade e facilidade de manutenção do sistema.

Inicialmente, foi realizada a escolha do framework Django para o desenvolvimento backend da aplicação, considerando suas funcionalidades integradas, segurança e suporte nativo aos padrões MVT (Model-View-Template) e ORM (Object-Relational Mapping). A equipe também realizou estudos sobre MVC, MVT e ORM, com o objetivo de compreender melhor a separação das responsabilidades do sistema e a comunicação entre interface, lógica e banco de dados.

Na modelagem do banco de dados, foram implementadas as chaves estrangeiras e constraints necessárias para garantir a integridade e o relacionamento correto entre as entidades do sistema. Também foram realizadas melhorias nos modelos relacionais e na organização das

tabelas, permitindo maior confiabilidade no armazenamento das informações.

No frontend, foram iniciados os estudos e definições dos layouts das páginas do sistema, buscando criar uma interface intuitiva, organizada e responsiva. Durante essa etapa, também foram realizados ajustes visuais e melhorias na experiência do usuário, visando facilitar a navegação e interação com a plataforma.

Além disso, foi iniciada a documentação técnica dos códigos e diagramas utilizados no projeto, registrando a estrutura do sistema, regras de negócio e organização das funcionalidades implementadas. Dessa forma, a Sprint 2 teve como principal objetivo consolidar a base técnica da aplicação e preparar a estrutura necessária para as próximas etapas do desenvolvimento.

10.3 SPRINT 3

Durante a Sprint 3, a equipe concentrou esforços na implementação prática das principais funcionalidades do sistema, bem como na consolidação da estrutura técnica e documental do projeto. Nesta etapa, houve avanço significativo tanto no desenvolvimento do backend quanto na integração entre as interfaces do sistema e o banco de dados.

Inicialmente, foram revisados e referenciados os modelos conceitual, lógico e físico do banco de dados, garantindo alinhamento entre a estrutura planejada e a implementação prática da aplicação. A partir desses modelos, foi realizada a implementação do banco de dados, incluindo tabelas, relacionamentos, chaves estrangeiras e constraints responsáveis por assegurar a integridade das informações armazenadas.

Também foram implementados os principais endpoints relacionados ao gerenciamento de reservas, permitindo o envio, recebimento e tratamento das informações entre frontend e backend. Durante essa etapa, foram aplicadas regras de negócio fundamentais do sistema, como controle de disponibilidade dos carrinhos, limitação de reservas por data e validações relacionadas ao fluxo de solicitação de pedidos.

No desenvolvimento das funcionalidades do sistema, foi implementado o cálculo automático do valor total do pedido, considerando os produtos selecionados e as regras definidas pela empresa. Além disso, foi iniciada a criação da funcionalidade de cálculo e envio de frete utilizando a plataforma Lalamove, visando automatizar parte da logística relacionada às entregas dos carrinhos.

No frontend, foram realizados avanços na integração com o backend, permitindo maior comunicação entre as páginas do sistema e o banco de dados. Também foi implementada a máscara de formatação automática para o campo de telefone, buscando melhorar a experiência

do usuário e reduzir erros de preenchimento durante o cadastro e realização das reservas.

Durante a Sprint 3, a equipe também realizou estudos e implementações relacionadas à experiência do usuário (UX), buscando tornar a navegação mais intuitiva, organizada e responsiva. Foram feitos ajustes visuais nas páginas, melhorias nos formulários e organização das informações apresentadas ao usuário, visando proporcionar uma experiência mais simples e eficiente.

Além das implementações técnicas, foram desenvolvidos elementos acadêmicos e documentais importantes para o projeto, como a ficha catalográfica, termo de aprovação, agradecimentos e documentação das tecnologias utilizadas no desenvolvimento do sistema. Também foram realizadas referências bibliográficas relacionadas às ferramentas e metodologias empregadas pela equipe.

Na parte de organização e versionamento, o GitHub continuou sendo utilizado para armazenar, organizar e controlar as alterações realizadas no projeto. A equipe também utilizou o ambiente localhost para execução e testes locais da aplicação, permitindo validação contínua das funcionalidades implementadas antes da integração definitiva.

Por fim, durante esta sprint, também foi iniciada a preparação da apresentação do simpósio, incluindo organização dos tópicos, estruturação da apresentação e definição das principais informações que seriam demonstradas sobre o sistema, suas funcionalidades e tecnologias utilizadas no desenvolvimento do projeto.

10.4 SPRINT 4

11. FUNCIONAMENTO DO SISTEMA

O funcionamento do sistema inicia quando o cliente acessa a plataforma web da empresa Oggi para consultar os carrinhos de sorvete disponíveis para locação. A aplicação foi desenvolvida para proporcionar um processo simples, intuitivo e organizado, permitindo que todo o gerenciamento das reservas seja realizado digitalmente.

Ao acessar o sistema, o cliente pode visualizar os carrinhos disponíveis, consultar os sabores de sorvete cadastrados e verificar as datas disponíveis para reserva. O sistema realiza automaticamente uma verificação de disponibilidade com base na quantidade de carrinhos cadastrados e nas reservas já confirmadas para determinada data.

Após selecionar os produtos e a data desejada, o cliente deve preencher um formulário contendo seus dados pessoais, como nome, telefone, endereço, e-mail e observações relacionadas ao evento. Essas informações são importantes para identificação do responsável

pela reserva e para facilitar a comunicação com a empresa.

Em seguida, o sistema realiza validações automáticas dos dados informados, garantindo que todos os campos obrigatórios estejam preenchidos corretamente. Caso existam inconsistências ou indisponibilidade de carrinhos, o sistema impede a continuidade da solicitação, evitando erros e conflitos de agendamento.

Com os dados validados, a solicitação de reserva é registrada no banco de dados com status pendente. Nesse momento, o sistema também gera automaticamente um link de contato via WhatsApp, permitindo que o cliente entre em contato diretamente com a empresa para continuidade do atendimento e confirmação oficial da reserva.

Na área administrativa, o administrador possui acesso às funcionalidades de gerenciamento do sistema. Por meio desse painel, é possível visualizar reservas cadastradas, confirmar ou cancelar pedidos, gerenciar carrinhos disponíveis, cadastrar produtos e acompanhar o histórico das solicitações realizadas pelos clientes.

O backend do sistema é responsável por processar todas as regras de negócio, incluindo controle de disponibilidade, validação de dados, gerenciamento de status das reservas e integração com o banco de dados. Já o frontend oferece uma interface responsiva e intuitiva para facilitar a navegação tanto em computadores quanto em dispositivos móveis.

O banco de dados armazena todas as informações do sistema, incluindo dados de clientes, reservas, produtos e carrinhos. A utilização do ORM do Django permite que essas informações sejam manipuladas de forma organizada e segura, reduzindo a complexidade das operações no banco de dados.

Além disso, o sistema foi desenvolvido seguindo princípios de segurança e proteção de dados, considerando práticas alinhadas à LGPD. O acesso administrativo é protegido por autenticação de usuários, e os dados são tratados de forma segura para garantir privacidade e integridade das informações.

Dessa forma, o sistema proporciona maior eficiência operacional, organização no processo de reservas, redução de erros manuais e melhoria no atendimento ao cliente, contribuindo para a modernização dos processos da empresa Oggi Sorvetes.

11.1 CONTROLE DE STATUS DE RESERVA

O sistema possui um mecanismo de controle de status responsável por acompanhar todo o ciclo de vida das reservas realizadas pelos clientes. Esse controle é fundamental para garantir a organização das solicitações, evitar conflitos de disponibilidade e permitir um gerenciamento eficiente dos carrinhos de sorvete.

Quando o cliente realiza uma solicitação de reserva pelo sistema, o pedido é automaticamente registrado com o status PENDENTE. Nesse estado, a solicitação já está armazenada no banco de dados, porém ainda depende da análise e confirmação do administrador. As reservas pendentes não ocupam oficialmente os carrinhos disponíveis, funcionando apenas como solicitações em análise.

Após a verificação da disponibilidade dos carrinhos, produtos selecionados e informações fornecidas pelo cliente, o administrador poderá alterar o status da reserva para CONFIRMADO. Quando isso acontece, o sistema passa a considerar o carrinho como ocupado na data escolhida, reduzindo automaticamente a disponibilidade para novas reservas. Essa regra é importante para impedir conflitos e evitar reservas acima da capacidade máxima permitida.

O sistema também permite que reservas sejam alteradas para o status CANCELADO. Esse cancelamento pode ocorrer por diferentes motivos, como desistência do cliente, indisponibilidade operacional, falha na confirmação via WhatsApp ou decisão administrativa. Ao cancelar uma reserva, o carrinho anteriormente associado volta a ficar disponível para novas solicitações.

Além dos status principais, o sistema utiliza regras de negócio para controlar a disponibilidade diária dos carrinhos. Apenas reservas confirmadas ocupam recursos do sistema. Dessa forma, mesmo que existam várias solicitações pendentes, somente os pedidos confirmados reduzem a quantidade de carrinhos disponíveis.

O administrador possui acesso a uma área administrativa onde pode visualizar todas as reservas cadastradas, consultar informações completas do cliente, verificar os produtos selecionados, acompanhar datas de eventos e realizar alterações de status conforme necessário. Esse gerenciamento facilita o controle operacional da empresa e melhora a organização das reservas.

O controle de status também auxilia na rastreabilidade das solicitações, permitindo identificar rapidamente quais reservas estão aguardando confirmação, quais já foram aprovadas e quais foram canceladas. Isso contribui para um processo mais seguro, organizado e confiável.

REFERÊNCIAS

ANEXOS

ANEXO 1 – Conferência de regra de negócio

Inicialmente, as regras relacionadas à capacidade de atendimento são tratadas principalmente nos métodos da classe Reserva. A RN01 (Capacidade diária) e a RN02 (Limite de confirmações) são atendidas pelo método `vagas_disponiveis`, que calcula a quantidade de carrinhos disponíveis com base no total de carrinhos ativos cadastrados (`Carrinho.objects.filter(status=True)`) e na quantidade de reservas com status “confirmado” para uma determinada data. Isso também cobre a RN06 (Indisponibilidade de data), pois quando o número de reservas confirmadas atinge o total de carrinhos (3), não há mais vagas disponíveis.

A RN03 (Reserva pendente não ocupa carrinho) e a RN04 (Apenas confirmadas ocupam carrinho) são respeitadas no mesmo método, já que apenas reservas com status 'confirmado' são consideradas na contagem de ocupação. Já a RN05 (Associação única de carrinho por data) é garantida de forma indireta pelo controle de vagas: como cada reserva confirmada consome uma vaga e o limite é fixo, evita-se que mais de um carrinho seja associado além da capacidade disponível.

A RN11 (Controle de concorrência) é atendida pelo método `pode_confirmar`, que verifica se ainda existem vagas antes de permitir a confirmação.

A RN10 (Alteração administrativa) é contemplada no método `clean`, que permite edição de reservas, inclusive quando já existem no banco (`self.pk`).

Em relação às regras financeiras, a RN13 (Valor da diária) é implementada na classe Carrinho, com o campo `preco_diaria`, e utilizada no método `taxa_aluguel`. Já a RN14 (Isenção da taxa por volume) está corretamente implementada: no método `taxa_aluguel`, caso o subtotal dos sorvetes (`subtotal_sorvetes`) seja maior ou igual a 300, a taxa de aluguel é zerada.

A RN07 (Confirmação via WhatsApp) é atendida pelo método `gerar_link_whatsapp`, que cria um link com os detalhes da reserva para envio direto ao cliente, estabelecendo esse canal como meio de confirmação.

A RN09 (Status válidos) define apenas três estados: “pendente”, “confirmado” e “cancelado”, com controle por reserva, mesmo contendo múltiplos carrinhos. Nesse ponto, o código já está corretamente alinhado, pois o atributo `STATUS_CHOICES` da classe Reserva utiliza exatamente esses três valores. Além disso, como o status está associado à reserva como um todo, o comportamento permanece consistente mesmo com a futura possibilidade de múltiplos carrinhos.

A RN12 (Múltiplas reservas por cliente na mesma data) permite que um cliente tenha mais de uma reserva no mesmo dia, desde que a capacidade máxima de carrinhos não seja

excedida. Nesse caso, o código já está compatível com a regra, pois não existe nenhuma validação bloqueando reservas duplicadas por cliente. Além disso, o controle de capacidade é feito pelos métodos `vagas_disponiveis` e `pode_confirmar`, garantindo que o limite de carrinhos disponíveis não seja ultrapassado.

ANEXO 2 – Estudo sobre a experiência de usuário

A experiência do usuário (UX) é um fator essencial no desenvolvimento de sistemas web, especialmente em aplicações voltadas ao público geral. Um sistema com boa usabilidade permite que o usuário execute suas tarefas de forma simples, intuitiva e eficiente, reduzindo erros e aumentando a satisfação. No contexto deste projeto, a melhoria da UX foi considerada um ponto fundamental para garantir que o processo de reserva de carrinhos e produtos ocorra de maneira clara e acessível.

Inicialmente, foi identificado que interfaces com excesso de etapas, falta de feedback visual e ausência de padronização podem dificultar a interação do usuário com o sistema. Dessa forma, busca-se implementar melhorias que tornassem o fluxo mais direto, como a organização visual dos produtos, utilização de botões de incremento e decremento de quantidade e a exibição clara das informações selecionadas pelo usuário.

Outro aspecto relevante é a implementação de feedbacks visuais, como mensagens de erro e confirmação, que auxiliam o usuário durante o preenchimento do formulário. Além disso, foram aplicadas validações de campos, como a formatação automática do telefone e verificação de campos obrigatórios, garantindo maior segurança e confiabilidade na inserção de dados.

Também foi considerado o conceito de responsividade, permitindo que o sistema seja utilizado em diferentes dispositivos, como smartphones e computadores. A adaptação da interface para diferentes tamanhos de tela contribui diretamente para a melhoria da experiência do usuário, tornando o sistema mais acessível e funcional.

Entende-se a usabilidade está diretamente relacionada à facilidade de aprendizado, eficiência, memorização, redução de erros e satisfação do usuário. Nesse sentido, as melhorias implementadas neste projeto seguem esses princípios, buscando oferecer uma interface simples, funcional e alinhada às boas práticas de design centrado no usuário.